

Castle Break

An A-Level Programming project for the controlled
assessment

Content page:

Table of Contents

3.1 problem identification	4
End user:	4
Demographic:	5
Computational thinking:	6
Stakeholders:	8
Research the problem:	9
Basic requirements:	11
Further Questions:	12
Analysis of results:	14
Detailed requirements:	15
Success Criteria:	15
Solution:	16
Hardware requirements:	17
3.2 design solution	18
Design specification:	19
Main Menu:	20
Settings menu:	23
Help Menu:	25
Winner Menu:	26
Pseudocode for the code:	28
Pseudocode Main menu:	28
Pseudocode Settings menu:	29
Pseudocode Help menu:	30
Pseudocode Winner menu:	31
Flow chart of Main menu:	36
Flow chart of Settings:	37
Winner Menu:	38
Help menu:	39
Usability:	40

Game variables table:	40
Test strategy:.....	43
Test strategy Main menu:	43
Test strategy Settings menu:	44
Test strategy Help menu:	44
Test strategy Winner menu:	45
Test strategy Main game :	45
3.3 Developing the solution:	47
Programming the game:.....	47
Finished Game code:	55
Testing:.....	61
Testing Main menu:	62
Testing Settings menu:.....	62
Testing Help menu:.....	63
Testing Winner menu:	63
Testing Main game :	63
Error occurred throughout the games development and testing:	64
Prototypes of the Game:	66
3.4 Evaluation:	68
Final look:.....	68
User testing:	72
Final solution:.....	73
Usability tested :	75
Success criteria testing:	75
Maintenance and development:	79

3.1 problem identification

End user:

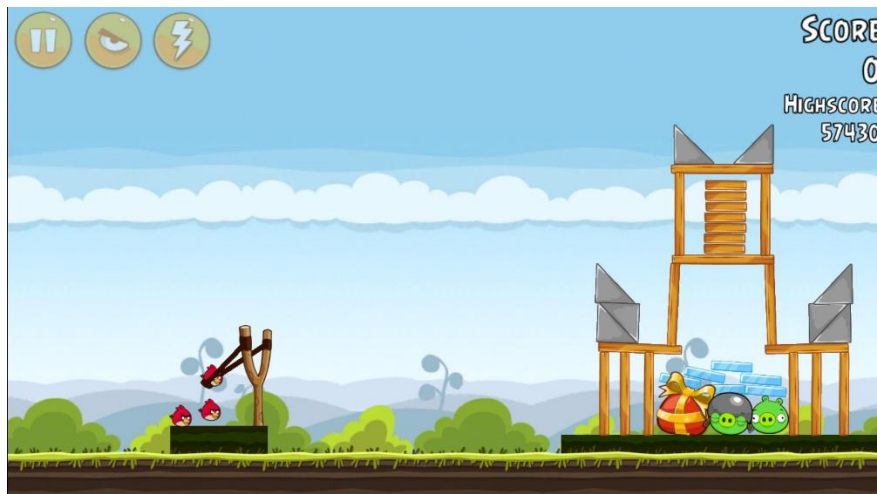
The aim user is people like students and children that are in group settings and have spare time to play against each other in a game that is competitive and fun. For example, siblings and people in pairs on a trip where they have nothing else to do and would want to play against each other.

Current method: Angry birds (2009-current)

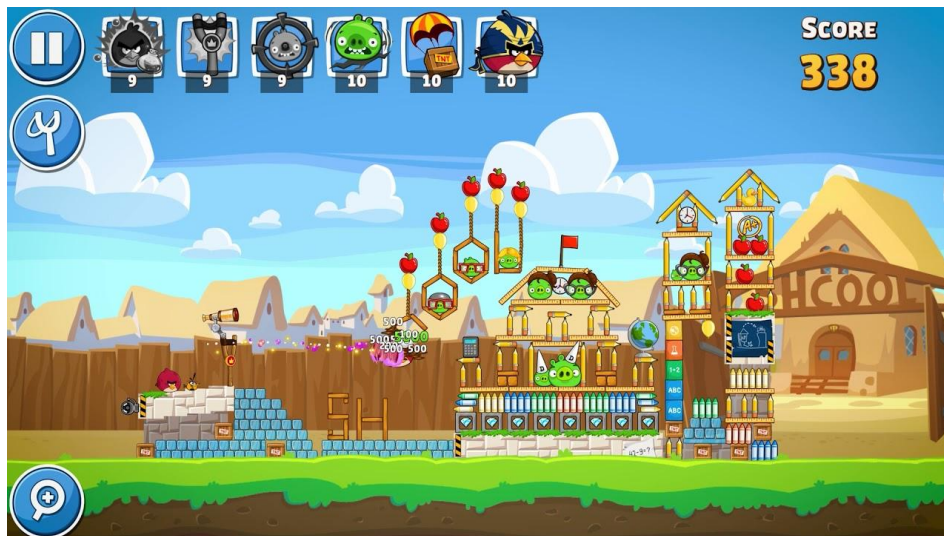
The base game of angry birds is a bird is flung from one side of the map to the other aimed to target a structure and to eliminate the green pigs. First developed in 2009 it was focused on mobile users and all ages with other improved versions made available after adding more types of birds and different seasonal looks. With the most recent version being made in 2015

My game will add a second user and allow the people to battle each other and attack the other structure which will have soldiers inside, there will be a catapult that Lunches medieval soldiers into the other personas structure until the structure is destroyed. My game will also simplify it turning it into a more enjoyable gameplay without the newly added choices of characters.

Here is the 2009 angry birds:



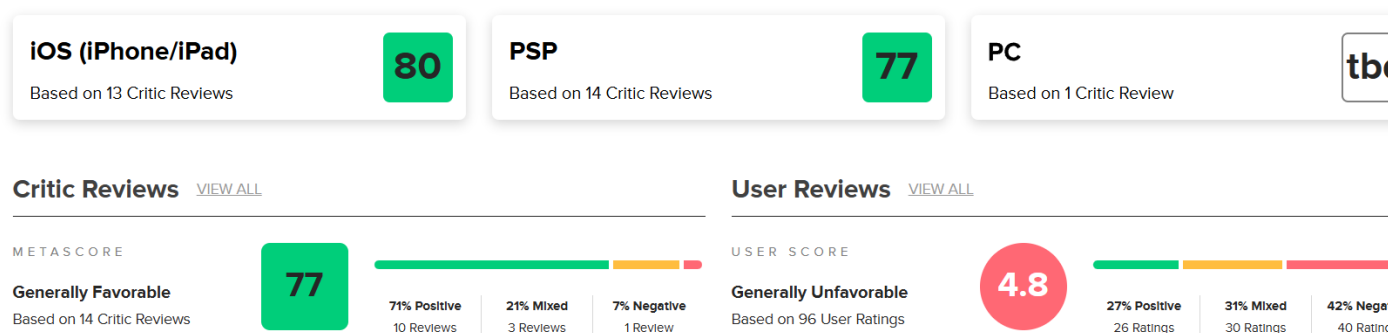
And one from a newer version:



The game is as can be seen single user and is highly complex aimed at destroying the characters inside (pigs).

Demographic:

Angry birds are mainly played by 35–44-year-olds 35% of those in the age group play at least once a month, compared to 22% of 18-22. Nearly all users use phones or tablets with 98% of 18 year olds having a mobile phone.



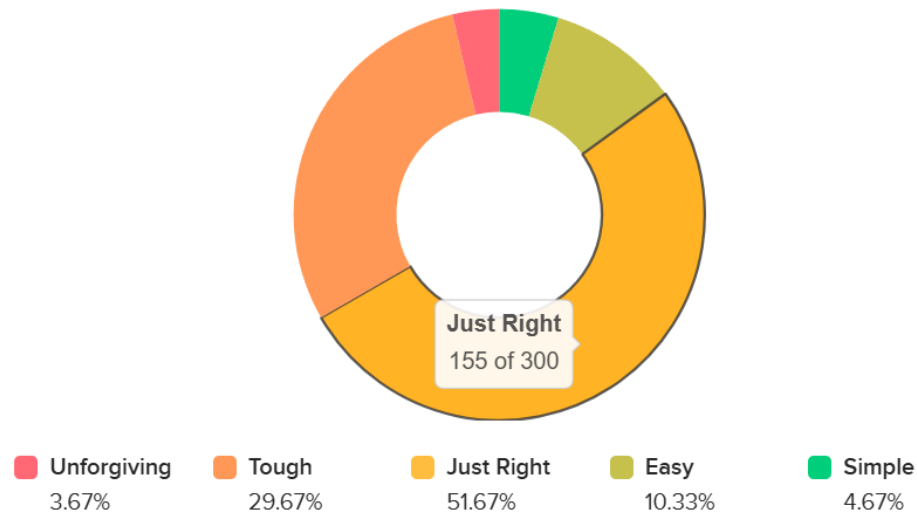
according to [Angry Birds Reviews - Metacritic](#)

As previously mentioned most of the demographic are ios users where most If not all users are positive on the other hand there when moving to psp and pc there are more negative ratings, specifically about the Design of the game which I plan to change.

Let me start by saying I really like Angry Birds (the original as well as Rio, Seasons, Star Wars and Space). Now I bought it for the PS3. The port itself is OK, the sticks are a bit tricky to use at the beginning, but it is not too bad. Graphics are just horrible. I expected some HD textures for the PS3, but I think the iPad version is much sharper. But definitely the worst flaw of this game would be the fact that there are only 4 worlds, while the iOS version already has 8. Still, this version is more expensive. Why? Not because of the textures. If you have an Android/iOS device, play it on that. Don't get this silly version. Still, it is playable, so if you don't have Android/iOS you can think of buying this.

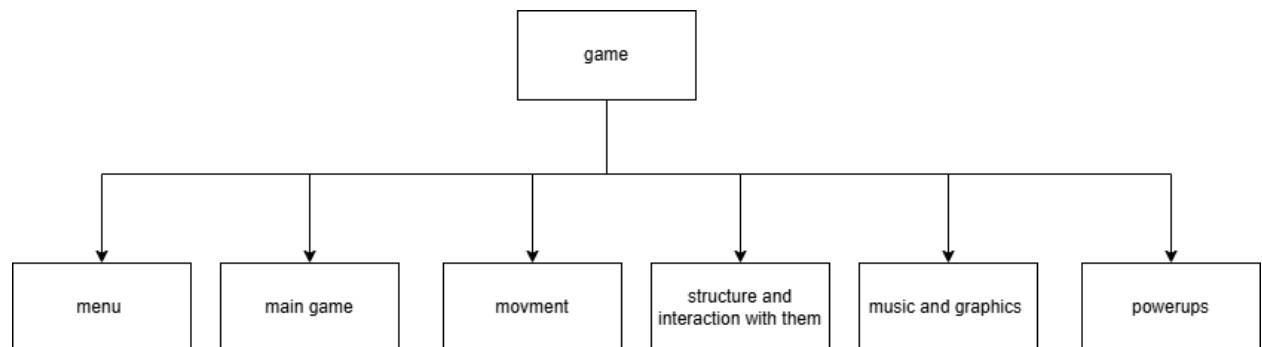
Game Stats for Angry Birds iOS (iPhone/iPad)

Difficulty Play Time Completion

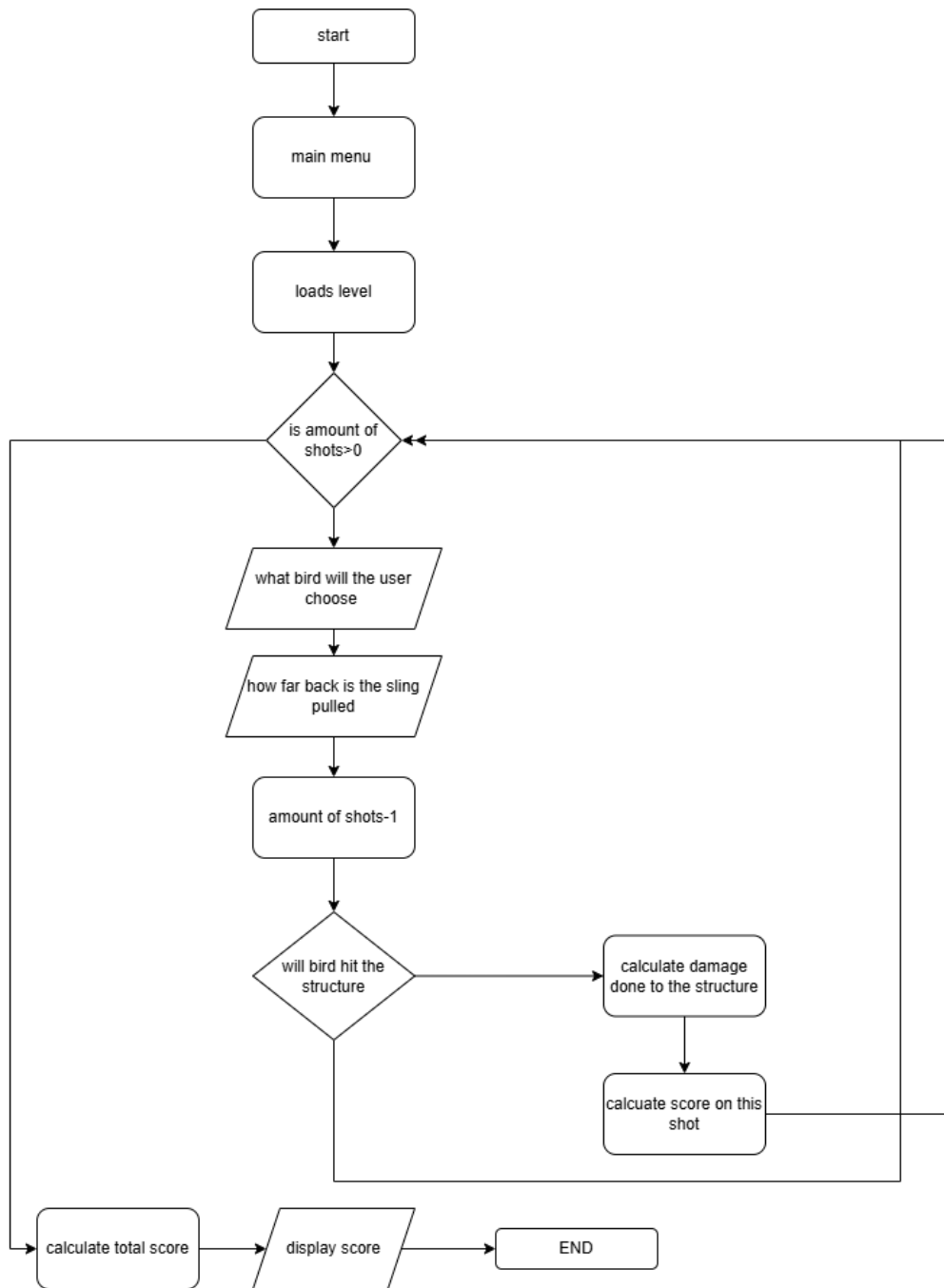


I Plan to similarly make this a just right difficulty if not easy rather than unforgivable or tough making the game more enjoyable.

Computational thinking:



The game can be broken down into 6 main sections that come together to make the whole game decomposition. It is done to simplify and make the game easier to make since you can focus on one part of the game at a time. Although not all parts will be interpreted into the game due to review of the stakeholders.



Flow chart of the current angry bird game;

Some limitations of the new proposed implementation are that for one it will only be accessed on a windows computer therefor a majority of the normal demographic won't be able to play. Therefor it may not have as large success as the original game. Second is that since its 2 players it will be turned based and somewhat repetitive after a while since custom build areas have not been implemented for the home base.

Problems	Possible solutions
----------	--------------------

Game is too repetitive	The game has the same goals the whole time with no change rather than one of settings, this can make it seem that the player is playing in a loop. The introduction of a powerups can allow for different gameplay.
Game is too complex	With so many options to choose from it can overwhelm the player on what the correct choice is. By removing the choice it allows for a more enjoyable gameplay without the worry of choice rather the focus on skill
Game is not competitive enough	By having the goal to be eliminate all the pigs the game becomes a puzzle solving game rather than a competitive. The game will have 2 players to compete against each other.

Stakeholders:

My stakeholders will be teenagers from my sixth form, which fit the demographic of having gaming as a hobby and in a need for a quick competitive gaming experience. Using their assistance I can develop the game in a pleasant and successful way in order to please the stakeholders and general public. I will then use their responses and about features and development to be integrated and adapted, changing the game of “angry birds” into something far more pleasant.

All 5 Stakeholders agreement of development as far:

Signature: _____

Research the problem:

A survey will be conducted to question the demographic of students on problems and requirements of possible users Along side that a deeper look into what improvements the users request from the current game.

Hardware and software requirements, the need for a mouse and keyboard to be able to interact with the game, along with a monitor so that the user can see the effects they are having. A device with python downloaded should be used since the program is in python.

To investigate end-user requirements the use of quizzes online will be used, where a questionnaire will be sent to 5, 3 of them males and 2 female to show that both males and females opinions be included along side this all are 17-18 year olds in education. The student are asked the same questions to see why they are unhappy with the game. They will answer the questions:

2. What things did you like about the angry birds games

5 Responses

ID ↑	Name	Responses
1	anonymous	The destroying buildings aspect
2	anonymous	overall game was good
3	anonymous	the gameplay itself was fun
4	anonymous	The sling shot
5	anonymous	the gameplay was good

3. What are things you disliked about the angry bird games

5 Responses

ID ↑	Name	Responses
1	anonymous	how repetitive it became after a while, although the buildings changed there was no change in the way things were done.
2	anonymous	how lonely it was to play
3	anonymous	the micro transactions
4	anonymous	There is no pc version
5	anonymous	so much bloat code

4. what improvements would you like to see if any improvements

5 Responses

ID ↑	Name	Responses
1	anonymous	A way more competitive way to play against your friends
2	anonymous	a way to get your friends involved and play against or with you
3	anonymous	a simpler version that's doesn't involve spending money
4	anonymous	a version on pc or laptop
5	anonymous	simpler code and smaller

From this investigation we can infer that a game with 2 player aspect will do good in the market, it will include the buildings and the slingshot but it will be two player so you can play with your friends it will also be on a pc due to the need to run pyhton3 that will also resolve the need for simplicity although the powerups don't seem to fit the gameplay as such the idea will be taken out.

Solution // requirements	Justification	Success
2 player	For competitiveness between players and	If both players can shoot
Building and sling shot	Keeping it original to the game	If both work as intended have physics to interact.
Simplicity	Able to run on all computers no matter the hardware, alongside this limits the lag of the game.	Not too much code
Code in pyhton3	Allowed to be ran on a pc	Program runs well on computers and can later be modified

Basic requirements:

- Blocks which are destructible
- A ball that bounces and is able to break the blocks
- The ability to pull back the ball and launch it
- The ability to run on a laptop or pc
- Noise//music
- 2 players
- Powerups
- Menus
- Ability to win the game

All 5 Stakeholders agreement of development as far:

Signature: _____

Further Questions:

These questions are based on hardware and preference of the stakeholders for the game

.A set of 16 questions was sent to all 5 stakeholders with the responses being:

	1	2	3	4	5
Are you satisfied with the basic requirements?	Yes	Yes	Yes	Yes	Yes
What device do you normally play on?	Laptop	Pc	Laptop	PC	laptop
What operating system do you use normally?	Windows 11	windows 10	10	windows 11	windows 11
How many blocks would be appropriate to include?	around 7	5	8	6	7
what colour should the ball be?	black or a colour that will stand out	black	blue	black	black
what colour should the background be?	white	white	white	white	white

what sounds if any would should be added?	when the block is broken	when the block is broken and when the ball bounces	no sound	the ball breaking a block	when the ball breaks a block
what button should settings menu be?	S	S	S	S	S
what button should main menu be?	TAB	M	M	TAB	M
what settings should be included?	The ball size and the gravity	the noise level, and the bouncines s of the walls	The size of the ball and the gravity	the size of the walls , noise level of the game , and the size of the ball	the level of noise and possible the amount of blocks
what button should leave the game be?	ESC	ESC	ESC	ESC	ESC
what size would the game best fit	Medium	Small	Medium	Large	Medium
what powerups would you like, if any?	No powerups	No powerups	Speed up and higher level of impact	None	Low gravity
what colour should the blocks be?	Black	Black	blue	black	black

What menu should the game boot up to?	Main menu	Main menu	Help menu	Main menu	Main menu
What button should Help be?	H	H	H	H	H

Analysis of results:

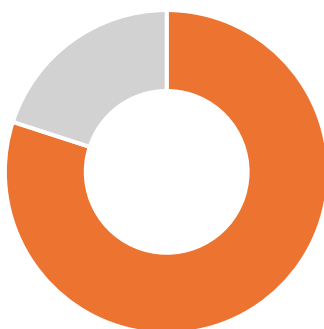
All stakeholders were in agreement with most if not all basic requirements with some minor editions.

The questions were aimed to assess how the game should look and be made, looking at what they the stakeholders ideas were for the game. With this I can assemble a list of design, mechanical and user specifications. Among the data pattens arose which I created pi charts for:

Medium accounts for the majority of 'what size would the game best fit'.



Black accounts for the majority of 'what colour should the blocks be?'.



Detailed requirements:

Design:

- Medium windows screen 1200, 700
- The ball will be black
- The blocks will be black
- The background will be white
- The walls will be black
- A noise will be made when the block is broken
- The ball will need to stay within the windows
- The block will need to stay within the boundaries of the window
- The size of the ball and walls will stay the same unless changed by the settings
- Menus will be created, one for help, main, settings and winner

Input:

- When the player drags back the will launch in the opposite direction
- When the player presses "TAB" the player will be given the main menu screen
- When the player presses "S" the player will be given the Settings menu screen
- when the player presses "H" the player will be given the Help menu screen
- when in settings the player can change a number which will directly affect the game
- when the player pulls back from the ball the game should show a small shape
- when the player presses "SPACE" the game will start

processes:

- when a ball is broken the countdown must update
- when all the shapes are broken the winner menu will pop up
- menus will show on screen when respected button is pressed
- The direction of the launch of the ball must be calculated

Output:

- Correct menus will show when required
- Noise when the block is broken
- The number of blocks should update each time the block is broken

Success Criteria:

Criteria	Explanation
Menus – (help, settings , main , game)	Should be able to move across menus and menus should work as intended with the ability to edit in the settings.

Ball movement	Should bounce and go in the direction intended the ball should also help velocity to see if when hit if it will break or not.
Polygon breaking	Should break when hit with a certain velocity and result in the breaking of it.
Difficulty	Game should be winnable but not too easy show present some challenge
Lag	Should not take too long to load up the game, or when hitting a shape(breakable polygon) it should not lag .
Responsiveness / Controls	The game should respond to user input and button presses responding in a timely manner.
Music/noise control	Music/noise should be able to be changed in terms of volume and if its on or off and when a noise activates it should do it in a timely order.
Powerups working	The ball should react when coming into contact the powerups and should do it without lag

All 5 Stakeholders agreement of development as far:

Signature: _____

Solution:

To create this game I use a library import of PyGames which helps with the game making process alongside Pymunk to assist with the 2-D simulation of physics.

While I make the game, I am limited by factors such as time, which may limit the ability to include some features. Alongside this I am also limited by my own knowledge as I have little experience in programming games as such some features may be too complicated to implement. Graphical limitations may also appear as I am not able to create complex designs for one due to the limitations of pygames but also due to the caution of time.

Hardware requirements:

Hardware//software	Justification
Keyboard	Able to start the game and edit the settings of the game
Mouse / touch screen	To be able to pull back the ball
Python 3	To be able to run the code and program the game
Monitor	To see the game play outputs of the game
PyMunk	Allows for easier 2D simulation in python
PyGame	Allows for the creation of the game
VsCode	Allows for the programing of the game and de-bugging
Windows 7 and above	Before windows 7 python will not run and therefor the game cannot be ran

All 5 Stakeholders agreement of development as far:

Signature: _____

3.2 design solution

Decomposing the new game is ideal for programming it as easily as possible, and making sure that all stakeholders are happy with the final product. I will create a number of stages that after following will create the game.

Stage 1 – design specification

Stage 2 – 5 – Game appearance and Menu designs

- All aimed at creating the look of the game and some mechanics

Stage 8 – pseudocode of the game

Stage 9 – flow charts

- These are aimed at creating it easier for me to program the game and how the code will work by itself

Design specification:

Aimed to determine the layout and appearance of the game using the stakeholders opinions as such a number of designs will be created, and the options are then rated by the stakeholders.

Design:

- Medium windows screen 1200, 700
- The ball will be black
- The blocks will be black
- The background will be white
- The walls will be black
- A noise will be made when the block is broken
- The ball will need to stay within the windows
- The block will need to stay within the boundaries of the window
- The size of the ball and walls will stay the same unless changed by the settings
- Menus will be created, one for help, main, settings and winner

Input:

- When the player drags back the will launch in the opposite direction
- When the player presses "TAB" the player will be given the main menu screen
- When the player presses "S" the player will be given the Settings menu screen
- when the player presses "H" the player will be given the Help menu screen
- when in settings the player can change a number which will directly affect the game
- when the player pulls back from the ball the game should show a small shape
- when the player presses "SPACE" the game will start

processes:

- when a ball is broken the countdown must update
- when all balls are broken the winner menu will pop up
- menus will show on screen when respected button is pressed
- The direction of the launch of the ball must be calculated

Output:

- Correct menus will show when required
- Noise when the block is broken
- The number of blocks should update each time the block is broken

All 5 Stakeholders agreement of development as far:

Signature: _____

Main Menu:

The initial window is the main menu where the player can choose to play the game, go to settings or to help.

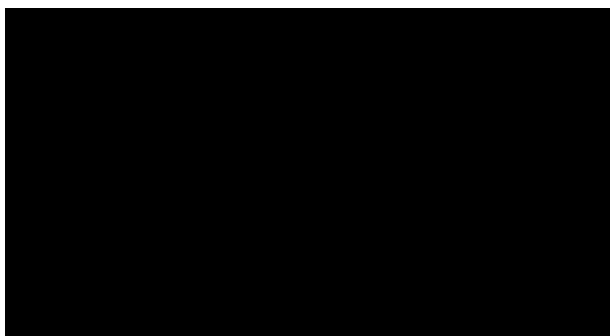
Properties of the main menu:

Object	Description	Justification
Text	The font of the text size and colour of the text “Arial” medium text and large for titles.	In order to communicate information clearly.
Input Keyboard press “SPACE”	When the button “SPACE” is pressed the game starts	So the user isn’t just sent to the game directly allows the user to move across menus
Input Keyboard press “S”	When the button “SPACE” on the keyboard is pressed the settings menu boots up	Allows to the user to change the game to their liking
Input Keyboard press “H”	When the button “H” on the keyboard is pressed the Help menu will pop up	Indicates how the game is played and the mechanics of it
Input Keyboard press “ESC”	When the button “ESC” on the keyboard is pressed the user exists from the game	In order to exit the game you can use the button “ESC”

Questionnaire:

There are 3 possible background designs for the main menu:

1.Black



2.Blue



3. White

What name would fit the Game best:

1. Castle Break
2. Block breaker
3. The Breaking

Result :

For the first quiz it was split 3 for black 2 for white with none voting for blue
The name chosen was 1 with 4 voting for Castel break and 1 for block breaker

Analysis:

It was decided that the name of the game would be Castel break with a majority ruling
and that the menus would be black.

Set up for the main menu:

Using the information it can be decided what the menu would look like.

Questionnaire 3:

What design is better for the main menu:

1.



2.

Castle break

press SPACE to start
press S for settings
press ESC to quite
Press H for help

3.

Castel Break

Press SPACE to Start

Press S for settings

Press H for Help

Press ESC to Quit

Results:

4 out of 5 voted for 1 with 1 voting for 2.

Analysis:

Its agreed that black will be used for the menu main with the first design setup

Settings menu:

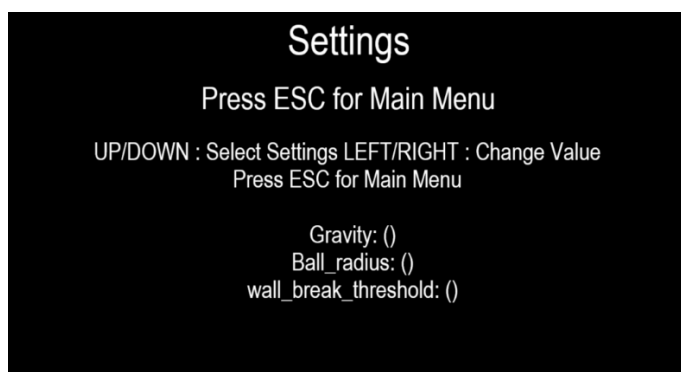
Settings is used to change how the settings of the game and changing in some ways how it's played.

Object	Description	Justification
Text	The font of the text size and colour of the text "Arial" medium text and large for titles. Some maybe be in White or Gray according to aesthetic choices	In order to communicate information clearly.
Input Keyboard press "UP/DOWN arrows"	When the button "UP" is pressed you select the setting that is above it allowing you edit Same with if you press "DOWN" but downwards both highlighting in a colour	in order to make it clear what setting is chosen and to allow the person to choose between settings
Input Keyboard press "LEFT/RIGHT arrows"	When the button "RIGHT" is pressed The setting increases by 20 and when the button "LEFT" it goes down by 20	In order to allow for changes depending the users wants
Input Keyboard press "ESC"	When the button "ESC" on the keyboard is pressed the user exists from the game	In order to exit the settings back to the main menu you can use the button "ESC"

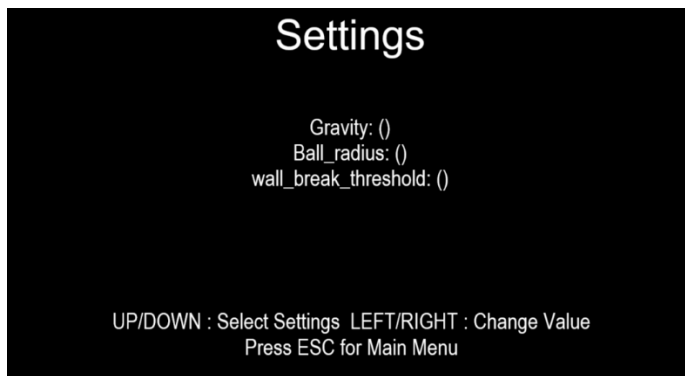
Questionnaire :

What is the preferred layout:

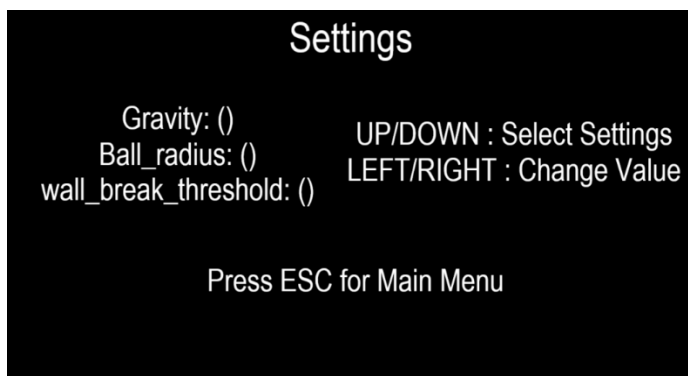
1.



2.



3.



Results:

3 out of 5 voted for no.2 while 2 voted for no.1

Analysis:

No.2 was the most voted design layout for the game and it is the one that will be used in the final game.

Help Menu:

In order to help those that do not know how to play the game a menu will be made specifically to guide the user.

Object	Description	Justification
Text	The font of the text size and colour of the text “Arial” medium text and large for titles.	In order to communicate information clearly.
Input Keyboard press “BACKSPACE”	When the button “BACKSPACE” is pressed the game starts	So the user isn’t just sent to the game directly allows the user to move across menus
Input Keyboard press “ESC”	When the button “ESC” on the keyboard is pressed the user will go back to the menu	In order to exit the menu to the main menu you can use the button “ESC”

Using the layout of the previous menu with a black background the help menu will look like this:



All 5 out of 5 shareholders agree to this layout of the help menu

Winner Menu:

Once all the blocks are gone this will be shown to allow the user to know they have won and to allow the user to play again

Object	Description	Justification
Text	The font of the text size and colour of the text “Arial” medium text and large for titles.	In order to communicate information clearly.
Input Keyboard press “BACKSPACE”	When the button “BACKSPACE” is pressed it will send him back to the main menu	So the user isn’t just sent to the game directly allows the user to move across menus
Input Keyboard press “ESC”	When the button “SPACE” on the keyboard is pressed the user will go back to the Game	Allows the user to go back to the game.

Using the layout of the previous menu with a black background the help menu will look like this:



All 5 out of 5 shareholders agree to this layout of the help menu

Amendment of the Specification:

Design:

- Medium windows screen 1200, 700
- The ball will be black
- The blocks will be black
- The background will be white
- The walls will be black
- A noise will be made when the block is broken
- The ball will need to stay within the windows
- The block will need to stay within the boundaries of the window
- The size of the ball and walls will stay the same unless changed by the settings
- Menus will be created, one for help, main, settings and winner
 - The background will be black
 - Writing will be white
 - A layout out writing will be placed in the middle
 - The writing font will be Arial

All 5 Stakeholders agreement of development as far:

Signature: _____

Pseudocode for the code:

The pseudocode is used to plan how the code will be laid out and how It will work planning the game and menus.

Pseudocode Main menu:

```
PROCEDURE runMainMenu
  WHILE TRUE
    FOR EACH event IN getEvents()
      IF event.type = "QUIT" THEN
        RETURN FALSE,
      ELSE IF event.type = "KEYDOWN" THEN
        IF event.key = "SPACE" THEN
          RETURN TRUE, GAME
        ELSE IF event.key = "S" THEN
          RETURN TRUE, SETTINGS
        ELSE IF event.key = "ESCAPE" THEN
          RETURN FALSE,
        ELSE IF event.key = "H" THEN
          RETURN TRUE, HELP
        ENDIF
      ENDIF
    NEXT event
  ENDWHILE
ENDPROCEDURE
```

This sets up the ineteraction of the main menu so if the user presses a button it will result in a change in variables.

```
font_large = pygame.font.SysFont('Arial', 50)
font_medium = pygame.font.SysFont('Arial', 36)
```

Previsouly defined font sizes and font.

```
title = font_large.render("Castle break", (255, 255, 255))
start_text = font_medium.render("Press SPACE to Start", (255, 255, 255))
settings_text = font_medium.render("Press S for Settings", (255, 255, 255))
quit_text = font_medium.render("Press ESC to Quit", (255, 255, 255))
help_text = font_medium.render("Press H for help", (255, 255, 255))
```

this will then be printed onto the screen for the user to see defining the colour and the font of the words.

Pseudocode Settings menu:

sets the default settings for the game with the variables and later allows for it to be edited.

```
settings =  
gravity": 900,  
ball_radius": 15,  
wall_break_threshold": 400
```

Used to change how the game works and plays by changing variables like the size of the ball and the gravity of the game.

PROCEDURE runSettings

```
    selectedSetting = 0  
    settingsOptions = getListOfSettings()
```

WHILE TRUE

```
    FOR EACH event IN getEvents()
```

```
        IF event.type = "QUIT" THEN
```

```
            RETURN FALSE,
```

```
        ELSE IF event.type = "KEYDOWN" THEN
```

```
            IF event.key = "ESCAPE" THEN
```

```
                RETURN TRUE, MAIN_MENU
```

```
            ELSE IF event.key = "UP" THEN
```

```
                selectedSetting = (selectedSetting - 1) MOD LENGTH(settingsOptions)
```

```
            ELSE IF event.key = "DOWN" THEN
```

```
                selectedSetting = (selectedSetting + 1) MOD LENGTH(settingsOptions)
```

```
            ELSE IF event.key = "LEFT" THEN
```

```
                settings[settingsOptions[selectedSetting]] = MAX(100,
```

```
settings[settingsOptions[selectedSetting]] - 20)
```

```
            ELSE IF event.key = "RIGHT" THEN
```

```
                settings[settingsOptions[selectedSetting]] =
```

```
settings[settingsOptions[selectedSetting]] + 20
```

```
            ELSE IF event.key = "H" THEN
```

```
                RETURN TRUE, HELP
```

```
            ENDIF
```

```
        ENDIF
```

```
    NEXT event
```

```
ENDWHILE
```

```
ENDPROCEDURE
```

Sets up the background process for the settings menu, allowing the user to change the settings to change and to select the settings that they want to change.

```
FOR i = 0 TO LENGTH(settingsItems) - 1
```

```

key = settingsItems[i].key
value = settingsItems[i].value
IF i = selectedSetting THEN
    color = (255, 255, 0)
ELSE
    color = (255, 255, 255)
ENDIF
text = renderText(fontMedium, key ,value, color)
NEXT i

```

This is used to change the colour if selected once the text is showed on screen.

Pseudocode Help menu:

If a player doesn't know how to play the game then they can press H in the main menu it will then mean they get sent to this menu.

This represents the back ground of the help menu:

```

PROCEDURE runHelpMenu
    WHILE TRUE
        FOR EACH event IN getEvents()
            IF event.type = "QUIT" THEN
                RETURN FALSE,
            ELSE IF event.type = "KEYDOWN" THEN
                IF event.key = "ESCAPE" THEN
                    RETURN TRUE, MAIN_MENU
                ELSE IF event.key = "BACKSPACE" THEN
                    RETURN TRUE, GAME
                ENDIF
            ENDIF
        NEXT event
    ENDWHILE
ENDPROCEDURE

```

To show this on screen the code would be;

```

title = font_large.render("Help", (255, 255, 255))
lines = [
    "Left Click & Drag on Ball to Shoot",
    "Break the red walls by hitting them",
    "Yellow text in Settings = currently selected option",
    "Press TAB to open Settings during the game",
    "Press ESC to return to Menu",
    "Press BACKSPACE to return to Game"
]

```

]

This shows the lines onto the screen for the user to read.

Pseudocode Winner menu:

When the player breaks all the blocks then this

```
WHILE TRUE
  FOR EACH event IN getEvents()
    IF event.type = "QUIT" THEN
      RETURN FALSE,
    ELSE IF event.type = "KEYDOWN" THEN
      IF event.key = "SPACE" THEN
        RETURN TRUE, GAME
      ELSE IF event.key = "ESCAPE" THEN
        RETURN TRUE, MAIN_MENU
      ENDIF
    ENDIF
  NEXT event
ENDWHILE
```

```
title = font_large.render("You Win!", True, (255, 255, 0))
restart_text = font_medium.render("Press SPACE to Play Again", True, (255, 255, 255))
menu_text = font_medium.render("Press ESC to Return to Menu", True, (255, 255, 255))
```

Main Game script:

This will be the main script for the game and how it will work.

```
PROCEDURE runGame
  space = createSpace()
  space.gravity = (0.0, -settings["gravity"])

  walls = [
    createSegment(space.staticBody, (0, 0), (0, HEIGHT), 5),
    createSegment(space.staticBody, (0, 0), (WIDTH, 0), 5),
    createSegment(space.staticBody, (WIDTH, 0), (WIDTH, HEIGHT), 5),
    createSegment(space.staticBody, (0, HEIGHT), (WIDTH, HEIGHT), 5)
  ]
  FOR EACH wall IN walls
    wall.friction = 1.0
    wall.elasticity = 0.9
    wall.collisionType = COLLTYPE_STATIC
    addToSpace(space, wall)
```

NEXT wall

```
polygons = createEmptyList()
balls = createEmptyList()
```

```
handler = addCollisionHandler(space, COLLTYPE_BALL, COLLTYPE_POLYGON)
handler.postSolve = handleWallCollision
```

```
polygons.append(createDynamicWall((500, 75), 200, 50, space, movable=True,
breakable=True))
polygons.append(createDynamicWall((500, 85), 200, 50, space, movable=True,
breakable=True))
polygons.append(createDynamicWall((500, 95), 200, 50, space, movable=True,
breakable=True))
polygons.append(createDynamicWall((500, 125), 200, 50, space, movable=True,
breakable=True))
balls.append(createBall((1100, 100), space))
```

```
mouseContact = createVec2d(0, 0)
selectedBall = createEmptyObject()
mouseStartPos = createVec2d(0, 0)
```

WHILE TRUE

```
FOR EACH event IN getEvents()
  IF event.type = "QUIT" THEN
    RETURN FALSE, MAIN_MENU
  ELSE IF event.type = "KEYDOWN" THEN
    IF event.key = "ESCAPE" THEN
      RETURN TRUE, MAIN_MENU
    ELSE IF event.key = "TAB" THEN
      RETURN TRUE, SETTINGS
    ELSE IF event.key = "H" THEN
      RETURN TRUE, HELP
    ENDIF
  ELSE IF event.type = "MOUSEBUTTONDOWN" AND event.button = 1 THEN
    p = createVec2d(event.pos[0], HEIGHT - event.pos[1])
    FOR EACH ball IN balls
      IF ball.pointQuery(p).distance < 0 THEN
        selectedBall = ball
        mouseStartPos = p
        BREAK
    ENDIF
  NEXT ball
  ELSE IF event.type = "MOUSEBUTTONUP" AND event.button = 1 THEN
    IF selectedBall IS NOT createEmptyObject() THEN
      p = createVec2d(event.pos[0], HEIGHT - event.pos[1])
      impulse = (mouseStartPos - p) * 10
```

```

    maxImpulse = 1000
    IF impulse.length > maxImpulse THEN
        impulse = normalize(impulse) * maxImpulse
    ENDIF
    applyImpulseAtLocalPoint(selectedBall.body, impulse)
    selectedBall = createEmptyObject()
    mouseStartPos = createVec2d(0, 0)
    ENDIF
    ELSE IF event.type = "MOUSEMOTION" THEN
        IF selectedBall IS NOT createEmptyObject() THEN
            mouseContact = createVec2d(event.pos[0], HEIGHT - event.pos[1])
        ENDIF
    ENDIF
NEXT event

IF NOT EXISTS poly IN polygons WHERE poly.breakable THEN
    RETURN TRUE, WINNER
ENDIF

stepPhysics(space, 1/60.0)

fillScreen("white")

FOR EACH polygon IN polygons
    drawPolygon(polygon, screen, lambda v: flipyv(v, HEIGHT))
NEXT polygon

FOR EACH wall IN walls
    p1 = flipyv(wall.a, HEIGHT)
    p2 = flipyv(wall.b, HEIGHT)
    drawLine(screen, p1, p2, 5, "black")
NEXT wall

FOR EACH ball IN balls
    drawBall(ball, screen, lambda v: flipyv(v, HEIGHT))
NEXT ball

IF mouseContact IS NOT createVec2d(0, 0) THEN
    p = flipyv(mouseContact, HEIGHT)
    drawCircle(screen, p, 3, "red")
ENDIF

helpText = renderText(fontMedium, "ESC: Menu | TAB: Settings", (100, 100, 100))
drawText(screen, helpText, (10, 10))

breakableLeft = count(polygons, lambda poly: poly.breakable)

```

```

        breakableText = renderText(fontMedium, "Breakable Walls Left: " & breakableLeft,
(200, 0, 0))
        drawText(screen, breakableText, (10, 50))

        updateScreen()
        waitForNextFrame(60)
    ENDWHILE
ENDPROCEDURE

```

Other procedures that are brought in into the main script:
Class to create the breakable walls

```

CLASS DynamicWall
    METHOD __init__(self, pos, length, height, space, movable=True, breakable=True,
mass=10.0)
        self.breakable = breakable
        self.destructionThreshold = settings["wall_break_threshold"]

        IF movable THEN
            moment = pm.momentForBox(mass, (length, height))
            self.body = pm.Body(mass, moment)
        ELSE
            self.body = pm.Body(bodyType=pm.Body.STATIC)
        ENDIF

        self.body.position = createVec2d(pos[0], pos[1])
        self.shape = pm.Poly.createBox(self.body, (length, height))
        self.shape.friction = 0.4
        self.shape.elasticity = 0.5
        self.shape.collisionType = COLLTYPE_POLYGON

        addToSpace(space, self.body)
        addToSpace(space, self.shape)
    END METHOD

    METHOD draw(self, screen, flipFunc)
        ps = [self.body.localToWorld(v) FOR EACH v IN self.shape.getVertices()]
        ps = [flipFunc(p) FOR EACH p IN ps]
        drawLines(screen, ps, 3, "black")
    END METHOD
END CLASS

```

Procedure to create the ball

```

PROCEDURE createBall(point, space, mass=1.0)
    radius = settings["ball_radius"]

```

```

moment = pm.momentForCircle(mass, 0, radius)
ballBody = pm.Body(mass, moment)
ballBody.position = createVec2d(point[0], point[1])

ballShape = pm.Circle(ballBody, radius)
ballShape.friction = 1.5
ballShape.elasticity = 0.9
ballShape.collisionType = COLLTYPE_BALL

addToSpace(space, ballBody)
addToSpace(space, ballShape)

RETURN ballShape
END PROCEDURE

PROCEDURE drawBall(ball, screen, flipFunc)
    body = ball.body
    p = flipFunc(body.position)
    r = ball.radius
    drawCircle(screen, p, r, "black", 2)
END PROCEDURE

```

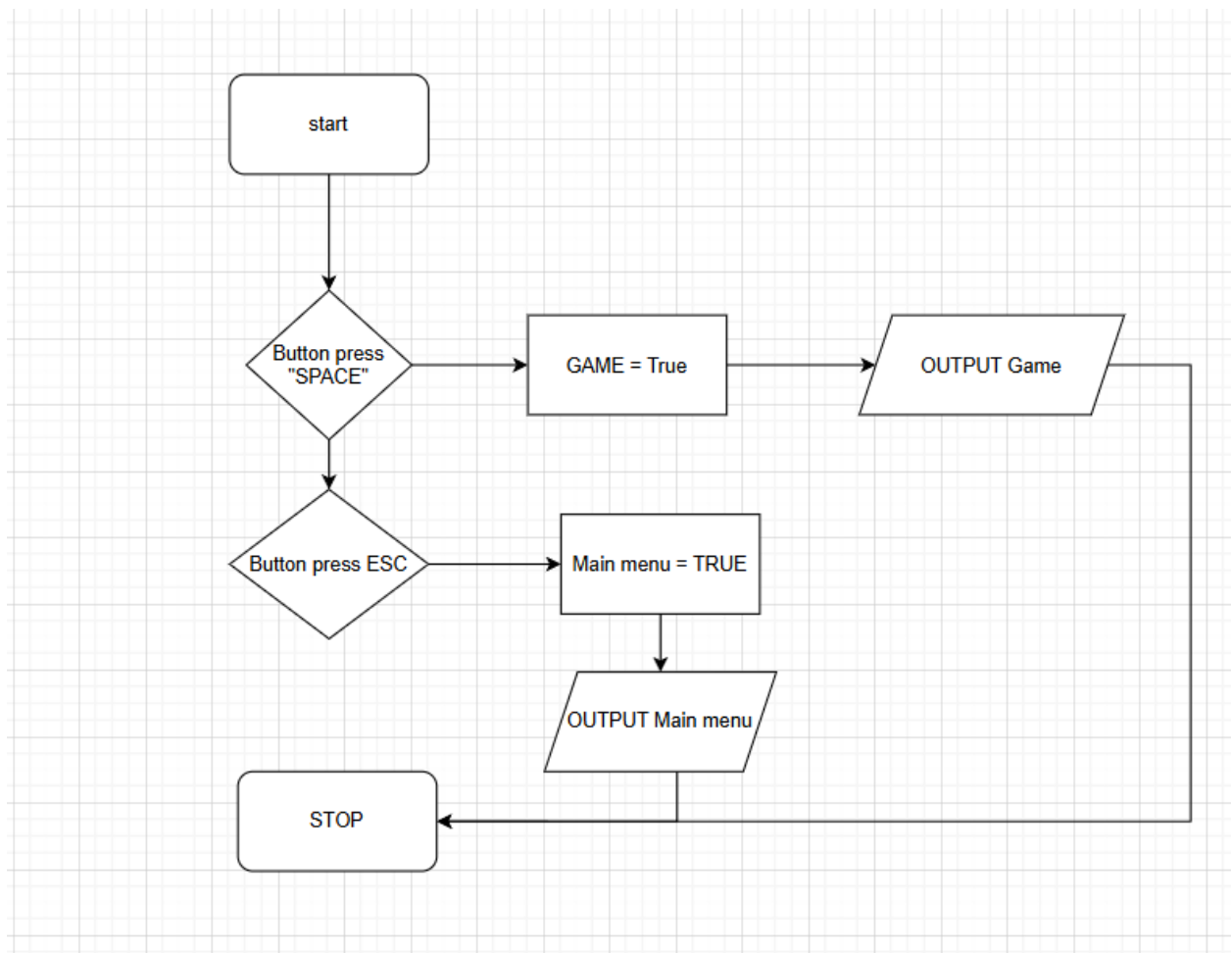
Flow chart of Main menu:



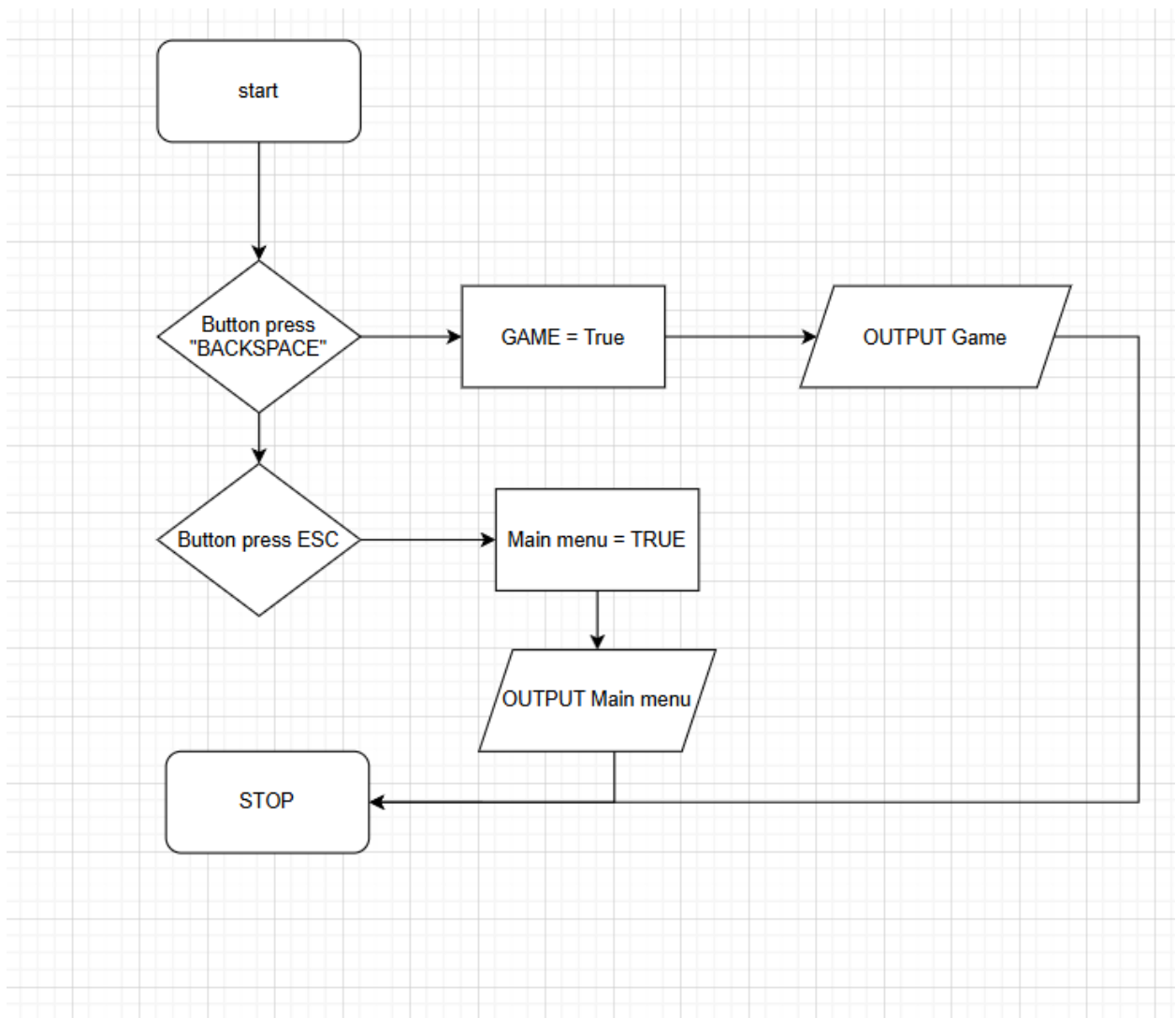
Flow chart of Settings:



Winner Menu:



Help menu:



Usability:

Features	Explanation
Assistance	Without this a new player would become confused and possibly not play the game
Simplicity	By making the game simple it could encourage different people of varying skills to play the game
Consistency	By having a similar game play it could make the game simpler and more fun
Design	By having an overall theme the player will associate the game with those colors

Game variables table:

Variable Name	Initial Value/Example	Description	Data Type
font_large	pygame.font.SysFont('Arial', 50)	Large font used for titles	Font Object
font_medium	pygame.font.SysFont('Arial', 36)	Medium font used for regular text	Font Object
title	Rendered text (e.g., "Castle Break")	Title text displayed on screens	Rendered Text Object
start_text	Rendered text ("Press SPACE to Start")	Start instruction text	Rendered Text Object

settings_text	Rendered text ("Press S for Settings")	Settings menu instruction text	Rendered Text Object
quit_text	Rendered text ("Press ESC to Quit")	Quit game instruction text	Rendered Text Object
help_text	Rendered text ("Press H for Help")	Help menu instruction text	Rendered Text Object
settings	<pre>{"gravity": 900, "ball_radius": 15, "wall_break_threshold": 400}</pre>	Game settings dictionary	Dictionary
selectedSetting	0	Index of the currently selected setting	Integer
settingsOptions	List of setting keys (["gravity", "ball_radius", "wall_break_threshold"])	List of adjustable settings	List (Array)
space	createSpace()	Physics space for the game	Space Object
walls	List of wall segments	Walls defining game boundaries	List

polygons	Empty list initially	List of dynamic wall objects	List
balls	Empty list initially	List of ball objects	List
handler	addCollisionHandler(...)	Collision handler for ball and wall collisions	Handler Object
mouseContact	createVec2d(0, 0)	Current mouse position during dragging	Vector2D Object
selectedBall	Empty object (createEmptyObject())	Ball currently selected by mouse	Object/None
mouseStartPos	createVec2d(0, 0)	Starting mouse position when clicking	Vector2D Object
COLLTYPE_STATIC	Constant/Enum value	Collision type for static objects	Constant/Enum
COLLTYPE_BALL	Constant/Enum value	Collision type for balls	Constant/Enum
COLLTYPE_POLYGON	Constant/Enum value	Collision type for dynamic walls	Constant/Enum
HEIGHT	Screen height	Used for position and flipping Y-axis	Integer
WIDTH	Screen width	Used for position and walls	Integer

screen	Screen surface	Main game display surface	Surface Object
helpText	Rendered small help info text	Help info during gameplay	Rendered Text Object
breakableLeft	Dynamic, based on walls left	Number of breakable walls remaining	Integer
lines (in help menu)	List of instruction strings	Lines explaining how to play the game	List
restart_text	Rendered text ("Press SPACE to Play Again")	Text shown after winning	Rendered Text Object
menu_text	Rendered text ("Press ESC to Return to Menu")	Text for returning to main menu after win	Rendered Text Object

Test strategy:

Test strategy Main menu:

Function tested	Justification for feature	Expected outcome
Button press "SPACE"	Make sure the starting of the button starts game	For the game to start
Button press "H"	Test that the button starts the help menu from the main menu	Help menu boots up
Button press "S"	Test that the button starts the settings menu from the main menu	The settings menu starts up
Button press "ESC"	Test that the button quitting the game	The game ends

Button press any other	Tests if any other then expected buttons are pressed	Nothing to happen
---------------------------	--	-------------------

Test strategy Settings menu:

Function tested	Justification for feature	Expected outcome
Button press “H”	Test that the button starts the help menu from the main menu	Help menu boots up
Button press “ESC”	Tests that the button results in goes back to the main menu	Main menu starts
Button press “UP”	Tests that when pressed that the setting selection selcts the one below	A change in colour from the one below to show that it is chosen
Button press “DOWN”	Tests that when pressed that the setting selection selcts the one above	A change in colour from the one above to show that it is chosen
Button press “LEFT”	Tests that when the button the number is go down by 20	The settings number to go down
Button press “RIGHT”	Tests that when the button the number is go up by 20	The settings number to go up
Button press any other	Tests if any other then expected buttons are pressed	Nothing to happen

Test strategy Help menu:

Function tested	Justification for feature	Expected outcome
Button press “ESC”	Tests that when pressed the main menu will appear	The main menu will pop up
Button press “BACKSPACE”	Tests that when pressed the game will start	The game to start
Button press any other	Tests if any other then expected buttons are pressed	Nothing to happen

Test strategy Winner menu:

Function tested	Justification for feature	Expected outcome
Button press "SPACE"	Tests that when pressed the main menu will appear	The main menu will pop up
Button press "BACKSPACE"	Tests that when pressed the game will start	The game to start
Button press any other	Tests if any other then expected buttons are pressed	Nothing to happen

Test strategy Main game :

Function tested	Justification for feature	Expected outcome
Button press "ESC"	Test that the button quitting the game	The game ends
Button press "TAB"	Test that the button qstarts the settings menu from the main menu	The settings menu starts up
Button press any other	Tests if any other then expected buttons are pressed	Nothing to happen
Ball action	Check that the ball will move in the inteded way	When the ball is clicked and draged downwards the ball will react in the oposit way.
polygons	Tests If the ball hits the breakable polygon	The polygon wall with the apropreate power the it will break
Walls	Tests if the ball is pulled back and hits the wall	The ball will bounce
counter	If a polygon is broken	The count should go down

All 5 Stakeholders agreement of development as far:

Signature: _____

3.3 Developing the solution:

Programming the game:

The game development can be split into different sections that aim to do different things. Each having its own step where it was created similar to the design.

1. First is the to import and define the stats and the constants of the game. Along side the creation of the display.

#game status :

Used to switch between menus easier with each menu having been assigned one status from 1-4 .

Constants are used for the different types of objects that could collid in the game.

```
import pygame
import pymunk as pm
from pymunk.vec2d import Vec2d

#Game states
MAIN_MENU = 0 #game state for main menu
SETTINGS = 1 #game state for settings menu
GAME = 2 #game state for main game
WINNER = 3 #game state for winner screen
HELP = 4 # game state for help menu

#Constants used for categorize different types of objects for collision handling
COLLTYPE_DEFAULT = 0
COLLTYPE_MOUSE = 1
COLLTYPE_BALL = 2
COLLTYPE_POLYGON = 3
COLLTYPE_STATIC = 4

#Initialize pygame and create screen
pygame.init()
WIDTH, HEIGHT = 1200, 700
screen = pygame.display.set_mode((WIDTH, HEIGHT))
clock = pygame.time.Clock()
font_large = pygame.font.SysFont('Arial', 50)
font_medium = pygame.font.SysFont('Arial', 36)
```

2. Defining the breakable polygons and how to load it:

This part of the code sets how the breakable polygons will appear and how it will work in the game space along side this is defines the creatin of the breakable polygon for easier later programming.

```
class BreakableBlocks: #Class for dynamic walls
    def __init__(self, pos, length, height, space, movable=True, breakable=True, mass=10.0): #sets up wall properties
        self.breakable = breakable
        self.destruction_threshold = settings["wall_break_threshold"]

        if movable:
            moment = pm.moment_for_box(mass, (length, height))
            self.body = pm.Body(mass, moment)
        else:
            self.body = pm.Body(body_type=pm.Body.STATIC)

        self.body.position = Vec2d(*pos) #sets the position of the wall
        self.shape = pm.Poly.create_box(self.body, (length, height)) #sets the shape of the wall
        self.shape.friction = 0.4 #friction of the wall
        self.shape.elasticity = 0.5 #bounciness of the wall
        self.shape.collision_type = COLLTYPE_POLYGON #collision type of the wall
        space.add(self.body, self.shape) #adds the wall to the space in pymunk

    def draw(self, screen, flip_func): #function to draw the wall
        ps = [self.body.local_to_world(v) for v in self.shape.get_vertices()]
        ps = [flip_func(p) for p in ps]
        pygame.draw.lines(screen, pygame.Color("black"), True, ps, 3)
```

3. Defining the ball and how to load it:

Like above a new function for the creation of the ball is used this allows the section to be called in the main running sequence, It designs all that is needed.

```
def create_ball(point, space, mass=1.0): #function to create a ball
    radius = settings["ball_radius"] #sets the radius of the ball
    moment = pm.moment_for_circle(mass, 0, radius) #sets the moment of inertia of the ball
    ball_body = pm.Body(mass, moment) #sets the mass of the ball
    ball_body.position = Vec2d(*point) #sets the position of the ball
    ball_shape = pm.Circle(ball_body, radius) #sets the shape of the ball
    ball_shape.friction = 1.5 #ball friction
    ball_shape.elasticity = 0.9 #ball bounciness
    ball_shape.collision_type = COLLYTYPE_BALL
    space.add(ball_body, ball_shape) #adds the ball to the space in pymunk
    return ball_shape

def draw_ball(ball, screen, flip_func): #function to draw the ball
    body = ball.body #sets the body of the ball
    p = flip_func(body.position) #sets the position of the ball
    r = ball.radius #sets the radius of the ball
    pygame.draw.circle(screen, pygame.Color("black"), p, int(r), 2) #draws the ball
```

4. Creating the polygon and ball interactions:

In order to create a seamless breaking of the polygons and allow the game to verify that the polygon is going to break , threshold the function is created

```
def handle_wall_collision(arbiter, space, data): #function to handle wall collision
    if (arbiter.shapes[0].collision_type == COLLYTYPE_BALL and #checks if the first shape is a ball
        arbiter.shapes[1].collision_type == COLLYTYPE_POLYGON): #checks if the second shape is a polygon

        impulse = arbiter.total_impulse.length #gets the impulse of the collision (impulse is the force of the collision)
        wall_shape = arbiter.shapes[1] #gets the shape of the wall

        for wall in polygons: #loops through all the polygons in order to check if the wall is breakable and if the impulse is greater than the destruction threshold
            if hasattr(wall, 'shape') and wall.shape == wall_shape and wall.breakable:
                if impulse > wall.destruction_threshold:
                    space.remove(wall.shape, wall.body)
                    polygons.remove(wall)
                    break
```

5. Sets up the main menu:

Following the preset design from 3.2 adding all the features talked about with screenshots of the in game menu

```
def run_main_menu(): #function to run the main menu and what happens when the user presses a specific button
    while True:
        for event in pygame.event.get():
            if event.type == pygame.QUIT: #
                return False, None
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_SPACE:
                    return True, GAME
                elif event.key == pygame.K_s:
                    return True, SETTINGS
                elif event.key == pygame.K_ESCAPE:
                    return False, None
                elif event.key == pygame.K_h:
                    return True, HELP

        screen.fill(pygame.Color("black"))
        #draws the main menu
        title = font_large.render("Castle break", True, (255, 255, 255))
        start_text = font_medium.render("Press SPACE to Start", True, (255, 255, 255))
        settings_text = font_medium.render("Press S for Settings", True, (255, 255, 255))
        quit_text = font_medium.render("Press ESC to Quit", True, (255, 255, 255))
        help_text = font_medium.render("Press H for help", True, (255, 255, 255))
        #draws the text on the screen
        screen.blit(title, (WIDTH//2 - title.get_width()//2, HEIGHT//4))
        screen.blit(start_text, (WIDTH//2 - start_text.get_width()//2, HEIGHT//2))
        screen.blit(settings_text, (WIDTH//2 - settings_text.get_width()//2, HEIGHT//2 + 50))
        screen.blit(quit_text, (WIDTH//2 - quit_text.get_width()//2, HEIGHT//2 + 100))
        screen.blit(help_text, (WIDTH//2 - help_text.get_width()//2, HEIGHT//2 + 150))

        pygame.display.flip()
        clock.tick(60)
```

Castle break

Press SPACE to Start
Press S for Settings
Press ESC to Quit
Press H for help

6. Set up of the help menu:

Following the preset design from 3.2 adding all the features talked about with screenshots of the in game menu

```
def run_help_menu():
    while True:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                return False, None
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_ESCAPE:
                    return True, MAIN_MENU
                elif event.key == pygame.K_BACKSPACE:
                    return True, GAME # Return to game if BACKSPACE pressed

        screen.fill(pygame.Color("black"))

        # Draw help content
        title = font_large.render("Help", True, (255, 255, 255))
        lines = [
            "Left Click & Drag on Ball to Shoot",
            "Break the red walls by hitting them",
            "Yellow text in Settings = currently selected option",
            "Press TAB to open Settings during the game",
            "Press ESC to return to Menu",
            "Press BACKSPACE to return to Game"
        ]

        screen.blit(title, (WIDTH//2 - title.get_width()//2, HEIGHT//6))
        for i, line in enumerate(lines):
            text = font_medium.render(line, True, (200, 200, 200))
            screen.blit(text, (WIDTH//2 - text.get_width()//2, HEIGHT//3 + i * 40))

        pygame.display.flip()
        clock.tick(60)
```

Help

Left Click & Drag on Ball to Shoot
Break the red walls by hitting them
Yellow text in Settings = currently selected option
Press TAB to open Settings during the game
Press ESC to return to Menu
Press BACKSPACE to return to Game

7. Set up of the settings menu:

Following the preset design from 3.2 adding all the features talked about with screenshots of the in game menu

```
def run_settings(): #function to run the settings menu
    selected_setting = 0
    settings_options = list(settings.keys())

    while True:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                return False, None
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_ESCAPE:
                    return True, MAIN_MENU
                elif event.key == pygame.K_UP:
                    selected_setting = (selected_setting - 1) % len(settings_options)
                elif event.key == pygame.K_DOWN:
                    selected_setting = (selected_setting + 1) % len(settings_options)
                elif event.key == pygame.K_LEFT:
                    settings[settings_options[selected_setting]] = max(100, settings[settings_options[selected_setting]] - 20)
                elif event.key == pygame.K_RIGHT:
                    settings[settings_options[selected_setting]] = settings[settings_options[selected_setting]] + 20
                elif event.key == pygame.K_h:
                    return True, HELP

        screen.fill(pygame.Color("black"))
        #draws the settings menu
        title = font_large.render("Settings", True, (255, 255, 255))
        screen.blit(title, (WIDTH//2 - title.get_width()//2, HEIGHT//6))
        #draws the title of the settings menu
        for i, (key, value) in enumerate(settings.items()):
            color = (255, 255, 0) if i == selected_setting else (255, 255, 255)
            text = font_medium.render(f"{key}: {value}", True, color)
            screen.blit(text, (WIDTH//2 - text.get_width()//2, HEIGHT//3 + i*40))
        #draws the settings options on the screen
        help_text = font_medium.render("UP/DOWN: Select setting | LEFT/RIGHT: Change value", True, (200, 200, 200))
        back_text = font_medium.render("Press ESC to return to menu", True, (200, 200, 200))
        #draws the help text and back text on the screen
        screen.blit(help_text, (WIDTH//2 - help_text.get_width()//2, HEIGHT - 100))
        screen.blit(back_text, (WIDTH//2 - back_text.get_width()//2, HEIGHT - 50))

        pygame.display.flip()
        clock.tick(60) #sets the frame rate to 60 FPS
```

Settings

gravity: 900
ball_radius: 15
wall_break_threshold: 400

UP/DOWN: Select setting | LEFT/RIGHT: Change value
Press ESC to return to menu

8. set up of the winner menu:

Following the preset design from 3.2 adding all the features talked about

```
def run_winner_menu(): #creates the winner menu
    #this function is called when the player wins the game
    while True:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                return False, None
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_SPACE:
                    return True, GAME
                elif event.key == pygame.K_ESCAPE:
                    return True, MAIN_MENU

        screen.fill(pygame.Color("black"))
        #draws the winner menu
        title = font_large.render("You Win!", True, (255, 255, 0))
        restart_text = font_medium.render("Press SPACE to Play Again", True, (255, 255, 255))
        menu_text = font_medium.render("Press ESC to Return to Menu", True, (255, 255, 255))
        #draws the title and text on the screen
        screen.blit(title, (WIDTH//2 - title.get_width()//2, HEIGHT//3))
        screen.blit(restart_text, (WIDTH//2 - restart_text.get_width()//2, HEIGHT//2))
        screen.blit(menu_text, (WIDTH//2 - menu_text.get_width()//2, HEIGHT//2 + 50))

        pygame.display.flip()
        clock.tick(60) #sets the frame rate to 60 FPS
```

You Win!

Press SPACE to Play Again
Press ESC to Return to Menu

Now the main script for the game starts, bringing all the functions and allowing it to run the game

9. the start of the main game code:

using a function allows for the game state to switch into the game or into a menu seamlessly , this main game brings all the functions from before and also draws all the writing.

```
def run_game():
    # Initialize physics space with current settings
    space = pm.Space()
    space.gravity = (0.0, -settings["gravity"])

    # Create static walls
    walls = [
        pm.Segment(space.static_body, (0, 0), (0, HEIGHT), 5),
        pm.Segment(space.static_body, (0, 0), (WIDTH, 0), 5),
        pm.Segment(space.static_body, (WIDTH, 0), (WIDTH, HEIGHT), 5),
        pm.Segment(space.static_body, (0, HEIGHT), (WIDTH, HEIGHT), 5),
    ]
    for wall in walls: #sets the properties of the walls
        wall.friction = 1.0 #friction of the wall
        wall.elasticity = 0.9 #bounciness of the wall
        wall.collision_type = COLLTYPE_STATIC #collision type of the wall
        space.add(wall)

    # Game objects
    global polygons
    polygons = []
    balls = []

    # Collision handler
    handler = space.add_collision_handler(COLLTYPE_BALL, COLLTYPE_POLYGON)
    handler.post_solve = handle_Polygon_collision
```

```

# Game state for running the game
mouse_contact = None
selected_ball = None
mouse_start_pos = None

while True:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            return False, None
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_ESCAPE:
                return True, MAIN_MENU
            elif event.key == pygame.K_TAB:
                return True, SETTINGS
            elif event.key == pygame.K_h:
                return True, HELP
        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            p = Vec2d(event.pos[0], HEIGHT - event.pos[1])
            for ball in balls:
                if ball.point_query(p).distance < 0:
                    selected_ball = ball
                    mouse_start_pos = p
                    break
            elif event.type == pygame.MOUSEBUTTONUP and event.button == 1:
                if selected_ball:
                    p = Vec2d(event.pos[0], HEIGHT - event.pos[1])
                    impulse = (mouse_start_pos - p) * 10
                    max_impulse = 1000
                    if impulse.length > max_impulse:
                        impulse = impulse.normalized() * max_impulse
                    selected_ball.body.apply_impulse_at_local_point(impulse)
                    selected_ball = None
                    mouse_start_pos = None
            elif event.type == pygame.MOUSEMOTION:
                if selected_ball:
                    mouse_contact = Vec2d(event.pos[0], HEIGHT - event.pos[1])

```

```

# Check win condition - no breakable polygons left
if not any(poly.breakable for poly in polygons):
    return True, WINNER

# Physics step
space.step(1/60.0)

# Drawing
screen.fill(pygame.Color("white"))

# Draw polygons
for polygon in polygons:
    polygon.draw(screen, lambda v: flipyv(v, HEIGHT))

# Draw walls
for wall in walls:
    p1 = flipyv(wall.a, HEIGHT)
    p2 = flipyv(wall.b, HEIGHT)
    pygame.draw.lines(screen, pygame.Color("black"), False, [p1, p2], 5)

# Draw balls
for ball in balls:
    draw_ball(ball, screen, lambda v: flipyv(v, HEIGHT))

# Draw mouse contact
if mouse_contact:
    p = flipyv(mouse_contact, HEIGHT)
    pygame.draw.circle(screen, pygame.Color("red"), p, 3)

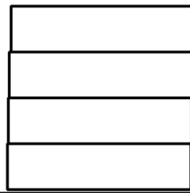
# Draw help text
help_text = font_medium.render("ESC: Menu | TAB: Settings", True, (100, 100, 100))
screen.blit(help_text, (10, 10))
# Draw remaining breakable polygons counter
breakable_left = sum(1 for poly in polygons if poly.breakable)
breakable_text = font_medium.render(f"Breakable Walls Left: {breakable_left}", True, (200, 0, 0))
screen.blit(breakable_text, (10, 50))

pygame.display.flip()
clock.tick(60) #sets the frame rate to 60 FPS

```

This is how the main game looks

ESC: Menu | TAB: Settings
Breakable Walls Left: 4



Finished Game code:

```
1  import pygame
2  import pymunk as pm
3  from pymunk.vec2d import Vec2d
4
5  #Game states
6  MAIN_MENU = 0    #game state for main menu
7  SETTINGS = 1     #game state for settings menu
8  GAME = 2         #game state for main game
9  WINNER = 3       #game state for winner screen
10  HELP = 4        # game state for help menu
11
12  #Constants used for categorize different types of objects for collision handling
13  COLLYTYPE_DEFAULT = 0
14  COLLYTYPE_MOUSE = 1
15  COLLYTYPE_BALL = 2
16  COLLYTYPE_POLYGON = 3
17  COLLYTYPE_STATIC = 4
18
19  #Initialize pygame and create screen
20  pygame.init()
21  WIDTH, HEIGHT = 1200, 700
22  screen = pygame.display.set_mode((WIDTH, HEIGHT))
23  clock = pygame.time.Clock()
24  font_large = pygame.font.SysFont('Arial', 50)
25  font_medium = pygame.font.SysFont('Arial', 36)
26
27  #Game settings (can be modified in settings menu)
28  settings = {
29      "gravity": 900,
30      "ball_radius": 15,
31      "wall_break_threshold": 400
32  }
33
34  class BreakableBlocks: #Class for dynamic walls
35      def __init__(self, pos, length, height, space, movable=True, breakable=True, mass=10.0): #sets up wall properties
36          self.breakable = breakable
37          self.destruction_threshold = settings["wall_break_threshold"]
38
39          if movable:
40              moment = pm.moment_for_box(mass, (length, height))
41              self.body = pm.Body(mass, moment)
42          else:
43              self.body = pm.Body(body_type=pm.Body.STATIC)
44
45          self.body.position = Vec2d(*pos) #sets the position of the wall
46          self.shape = pm.Poly.create_box(self.body, (length, height)) #sets the shape of the wall
47          self.shape.friction = 0.4 #friction of the wall
48          self.shape.elasticity = 0.5 #bounciness of the wall
49          self.shape.collision_type = COLLYTYPE_POLYGON #collision type of the wall
50          space.add(self.body, self.shape) #adds the wall to the space in pymunk
51
52      def draw(self, screen, flip_func): #function to draw the wall
53          ps = [self.body.local_to_world(v) for v in self.shape.get_vertices()]
54          ps = [flip_func(p) for p in ps]
55          pygame.draw.lines(screen, pygame.Color("black"), True, ps, 3)
```

```

56
57 def create_ball(point, space, mass=1.0): #function to create a ball
58     radius = settings["ball_radius"] #sets the radius of the ball
59     moment = pm.moment_for_circle(mass, 0, radius) #sets the moment of inertia of the ball
60     ball_body = pm.Body(mass, moment) #sets the mass of the ball
61     ball_body.position = Vec2d(*point) #sets the position of the ball
62     ball_shape = pm.Circle(ball_body, radius) #sets the shape of the ball
63     ball_shape.friction = 1.5 #ball friction
64     ball_shape.elasticity = 0.9 #ball bounciness
65     ball_shape.collision_type = COLLYTYPE_BALL
66     space.add(ball_body, ball_shape) #adds the ball to the space in pymunk
67     return ball_shape
68
69 def draw_ball(ball, screen, flip_func): #function to draw the ball
70     body = ball.body #sets the body of the ball
71     p = flip_func(body.position) #sets the position of the ball
72     r = ball.radius #sets the radius of the ball
73     pygame.draw.circle(screen, pygame.Color("black"), p, int(r), 2) #draws the ball
74
75 def flipyv(v, screen_height): #function to flip the y coordinate of the ball
76     # This is used to convert pymunk coordinates to pygame coordinates
77     return int(v.x), int(-v.y + screen_height) #returns the flipped coordinates
78
79
80
81 def handle_Polygon_collision(arbiter, space, data): #function to handle wall collision
82     if (arbiter.shapes[0].collision_type == COLLYTYPE_BALL and #checks if the first shape is a ball
83         arbiter.shapes[1].collision_type == COLLYTYPE_POLYGON): #checks if the second shape is a polygon
84
85         impulse = arbiter.total_impulse.length #gets the impulse of the collision (impulse is the force of the collision)
86         wall_shape = arbiter.shapes[1] #gets the shape of the wall
87
88         for wall in polygons: #loops through all the polygons in order to check if the wall is breakable and if the impulse is greater than the destruction threshold
89             if hasattr(wall, 'shape') and wall.shape == wall_shape and wall.breakable:
90                 if impulse > wall.destruction_threshold:
91                     space.remove(wall.shape, wall.body)
92                     polygons.remove(wall)
93                     break
94
95 def run_main_menu(): #function to run the main menu and what happens when the user presses a specific button
96     while True:
97         for event in pygame.event.get():
98             if event.type == pygame.QUIT: #
99                 return False, None
100             elif event.type == pygame.KEYDOWN:
101                 if event.key == pygame.K_SPACE:
102                     return True, GAME
103                 elif event.key == pygame.K_s:
104                     return True, SETTINGS

```

```

103         elif event.key == pygame.K_s:
104             return True, SETTINGS
105         elif event.key == pygame.K_ESCAPE:
106             return False, None
107         elif event.key == pygame.K_h:
108             return True, HELP
109
110     screen.fill(pygame.Color("black"))
111     #draws the main menu
112     title = font_large.render("Castle break", True, (255, 255, 255))
113     start_text = font_medium.render("Press SPACE to Start", True, (255, 255, 255))
114     settings_text = font_medium.render("Press S for Settings", True, (255, 255, 255))
115     quit_text = font_medium.render("Press ESC to Quit", True, (255, 255, 255))
116     help_text = font_medium.render("Press H for help", True, (255, 255, 255))
117     #draws the text on the screen
118     screen.blit(title, (WIDTH//2 - title.get_width()//2, HEIGHT//4))
119     screen.blit(start_text, (WIDTH//2 - start_text.get_width()//2, HEIGHT//2))
120     screen.blit(settings_text, (WIDTH//2 - settings_text.get_width()//2, HEIGHT//2 + 50))
121     screen.blit(quit_text, (WIDTH//2 - quit_text.get_width()//2, HEIGHT//2 + 100))
122     screen.blit(help_text, (WIDTH//2 - quit_text.get_width()//2, HEIGHT//2 + 150))
123
124     pygame.display.flip()
125     clock.tick(60)
126
127 def run_help_menu():
128     while True:
129         for event in pygame.event.get():
130             if event.type == pygame.QUIT:
131                 return False, None
132             elif event.type == pygame.KEYDOWN:
133                 if event.key == pygame.K_ESCAPE:
134                     return True, MAIN_MENU
135                 elif event.key == pygame.K_BACKSPACE:
136                     return True, GAME # Return to game if BACKSPACE pressed
137
138         screen.fill(pygame.Color("black"))
139
140         # Draw help content
141         title = font_large.render("Help", True, (255, 255, 255))
142         lines = [
143             "Left Click & Drag on Ball to Shoot",
144             "Break the red walls by hitting them",
145             "Yellow text in Settings = currently selected option",
146             "Press TAB to open Settings during the game",
147             "Press ESC to return to Menu",
148             "Press BACKSPACE to return to Game"
149         ]
150
151         screen.blit(title, (WIDTH//2 - title.get_width()//2, HEIGHT//6))
152         for i, line in enumerate(lines):
153             text = font_medium.render(line, True, (200, 200, 200))

```

```

154         screen.blit(text, (WIDTH//2 - text.get_width()//2, HEIGHT//3 + i * 40))
155
156     pygame.display.flip()
157     clock.tick(60)
158
159     def run_settings(): #function to run the settings menu
160         selected_setting = 0
161         settings_options = list(settings.keys())
162
163         while True:
164             for event in pygame.event.get():
165                 if event.type == pygame.QUIT:
166                     return False, None
167                 elif event.type == pygame.KEYDOWN:
168                     if event.key == pygame.K_ESCAPE:
169                         return True, MAIN_MENU
170                     elif event.key == pygame.K_UP:
171                         selected_setting = (selected_setting - 1) % len(settings_options)
172                     elif event.key == pygame.K_DOWN:
173                         selected_setting = (selected_setting + 1) % len(settings_options)
174                     elif event.key == pygame.K_LEFT:
175                         settings[settings_options[selected_setting]] = max(100, settings[settings_options[selected_setting]] - 20)
176                     elif event.key == pygame.K_RIGHT:
177                         settings[settings_options[selected_setting]] = settings[settings_options[selected_setting]] + 20
178                     elif event.key == pygame.K_h:
179                         return True, HELP
180
181             screen.fill(pygame.Color("black"))
182             #draws the settings menu
183             title = font_large.render("Settings", True, (255, 255, 255))
184             screen.blit(title, (WIDTH//2 - title.get_width()//2, HEIGHT//6))
185             #draws the title of the settings menu
186             for i, (key, value) in enumerate(settings.items()):
187                 color = (255, 255, 0) if i == selected_setting else (255, 255, 255)
188                 text = font_medium.render(f"{key}: {value}", True, color)
189                 screen.blit(text, (WIDTH//2 - text.get_width()//2, HEIGHT//3 + i*40))
190             #draws the settings options on the screen
191             help_text = font_medium.render("UP/DOWN: Select setting | LEFT/RIGHT: Change value", True, (200, 200, 200))
192             back_text = font_medium.render("Press ESC to return to menu", True, (200, 200, 200))
193             #draws the help text and back text on the screen
194             screen.blit(help_text, (WIDTH//2 - help_text.get_width()//2, HEIGHT - 100))
195             screen.blit(back_text, (WIDTH//2 - back_text.get_width()//2, HEIGHT - 50))
196
197             pygame.display.flip()
198             clock.tick(60) #sets the frame rate to 60 FPS
199
200
201     def run_winner_menu(): #creates the winner menu
202         #this function is called when the player wins the game
203         while True:
204             for event in pygame.event.get():
205                 if event.type == pygame.QUIT:
206                     return False, None

```

```

206         return False, None
207     elif event.type == pygame.KEYDOWN:
208         if event.key == pygame.K_SPACE:
209             return True, GAME
210         elif event.key == pygame.K_ESCAPE:
211             return True, MAIN_MENU
212
213     screen.fill(pygame.Color("black"))
214     #draws the winner menu
215     title = font_large.render("You Win!", True, (255, 255, 0))
216     restart_text = font_medium.render("Press SPACE to Play Again", True, (255, 255, 255))
217     menu_text = font_medium.render("Press ESC to Return to Menu", True, (255, 255, 255))
218     #draws the title and text on the screen
219     screen.blit(title, (WIDTH//2 - title.get_width()//2, HEIGHT//3))
220     screen.blit(restart_text, (WIDTH//2 - restart_text.get_width()//2, HEIGHT//2))
221     screen.blit(menu_text, (WIDTH//2 - menu_text.get_width()//2, HEIGHT//2 + 50))
222
223     pygame.display.flip()
224     clock.tick(60) #sets the frame rate to 60 FPS
225
226 def run_game():
227     # Initialize physics space with current settings
228     space = pm.Space()
229     space.gravity = (0.0, -settings["gravity"])
230
231     # Create static walls
232     walls = [
233         pm.Segment(space.static_body, (0, 0), (0, HEIGHT), 5),
234         pm.Segment(space.static_body, (0, 0), (WIDTH, 0), 5),
235         pm.Segment(space.static_body, (WIDTH, 0), (WIDTH, HEIGHT), 5),
236         pm.Segment(space.static_body, (0, HEIGHT), (WIDTH, HEIGHT), 5),
237     ]
238     for wall in walls: #sets the properties of the walls
239         wall.friction = 1.0 #friction of the wall
240         wall.elasticity = 0.9 #bounciness of the wall
241         wall.collision_type = COLLTYPE_STATIC #collision type of the wall
242         space.add(wall)
243
244     # Game objects
245     global polygons
246     polygons = []
247     balls = []
248
249     # Collision handler
250     handler = space.add_collision_handler(COLLTYPE_BALL, COLLTYPE_POLYGON)
251     handler.post_solve = handle_Polygon_collision

```

```

252
253     # Create initial objects and if any are breakable and movable
254     polygons.append(BreakableBlocks((500, 75), 200, 50, space, movable=True, breakable=True))
255     polygons.append(BreakableBlocks((500, 85), 200, 50, space, movable=True, breakable=True))
256     polygons.append(BreakableBlocks((500, 95), 200, 50, space, movable=True, breakable=True))
257     polygons.append(BreakableBlocks((500, 125), 200, 50, space, movable=True, breakable=True))
258     balls.append(create_ball((1100, 100), space))
259
260     # Game state for running the game
261     mouse_contact = None
262     selected_ball = None
263     mouse_start_pos = None
264
265     while True:
266         for event in pygame.event.get():
267             if event.type == pygame.QUIT:
268                 return False, None
269             elif event.type == pygame.KEYDOWN:
270                 if event.key == pygame.K_ESCAPE:
271                     return True, MAIN_MENU
272                 elif event.key == pygame.K_TAB:
273                     return True, SETTINGS
274                 elif event.key == pygame.K_h:
275                     return True, HELP
276             elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
277                 p = Vec2d(event.pos[0], HEIGHT - event.pos[1])
278                 for ball in balls:
279                     if ball.point_query(p).distance < 0:
280                         selected_ball = ball
281                         mouse_start_pos = p
282                         break
283             elif event.type == pygame.MOUSEBUTTONUP and event.button == 1:
284                 if selected_ball:
285                     p = Vec2d(event.pos[0], HEIGHT - event.pos[1])
286                     impulse = (mouse_start_pos - p) * 10
287                     max_impulse = 1000
288                     if impulse.length > max_impulse:
289                         impulse = impulse.normalized() * max_impulse
290                     selected_ball.body.apply_impulse_at_local_point(impulse)
291                     selected_ball = None
292                     mouse_start_pos = None
293             elif event.type == pygame.MOUSEMOTION:
294                 if selected_ball:
295                     mouse_contact = Vec2d(event.pos[0], HEIGHT - event.pos[1])
296
297     # Check win condition - no breakable polygons left
298     if not any(poly.breakable for poly in polygons):
299         return True, WINNER
300
301     # Physics step
302     space.step(1/60.0)
303

```

```

303
304     # Drawing
305     screen.fill(pygame.Color("white"))
306
307     # Draw polygons
308     for polygon in polygons:
309         polygon.draw(screen, lambda v: flipyv(v, HEIGHT))
310
311     # Draw walls
312     for wall in walls:
313         p1 = flipyv(wall.a, HEIGHT)
314         p2 = flipyv(wall.b, HEIGHT)
315         pygame.draw.lines(screen, pygame.Color("black"), False, [p1, p2], 5)
316
317     # Draw balls
318     for ball in balls:
319         draw_ball(ball, screen, lambda v: flipyv(v, HEIGHT))
320
321     # Draw mouse contact
322     if mouse_contact:
323         p = flipyv(mouse_contact, HEIGHT)
324         pygame.draw.circle(screen, pygame.Color("red"), p, 3)
325
326     # Draw help text
327     help_text = font_medium.render("ESC: Menu | TAB: Settings", True, (100, 100, 100))
328     screen.blit(help_text, (10, 10))
329     #Draw remaining breakable polygons counter
330     breakable_left = sum(1 for poly in polygons if poly.breakable)
331     breakable_text = font_medium.render(f"Breakable Walls Left: {breakable_left}", True, (200, 0, 0))
332     screen.blit(breakable_text, (10, 50))
333
334     pygame.display.flip()
335     clock.tick(60) #sets the frame rate to 60 FPS
336
337
338 def main(): #main function to run the game
339     current_state = MAIN_MENU
340     running = True
341     # Main game loop
342     while running:
343         if current_state == MAIN_MENU:
344             running, next_state = run_main_menu()
345             current_state = next_state if next_state is not None else current_state
346         elif current_state == SETTINGS:
347             running, next_state = run_settings()
348             current_state = next_state if next_state is not None else current_state
349         elif current_state == GAME:
350             running, next_state = run_game()
351             current_state = next_state if next_state is not None else current_state
352         elif current_state == WINNER:
353             running, next_state = run_winner_menu()
354
355     running, next_state = run_game()
356     current_state = next_state if next_state is not None else current_state
357     elif current_state == WINNER:
358         running, next_state = run_winner_menu()
359         current_state = next_state if next_state is not None else current_state
360     elif current_state == HELP:
361         running, next_state = run_help_menu()
362         current_state = next_state if next_state is not None else current_state
363
364     pygame.quit()
365
366 if __name__ == "__main__": #created to run the game and start from a specific function
367     main()

```

Testing:

Testing Main menu:

Function tested	Justification for feature	Expected outcome	Result
Button press "SPACE"	Make sure the starting of the button starts game	For the game to start	The game starts
Button press "H"	Test that the button starts the help menu from the main menu	Help menu boots up	The help menu starts
Button press "S"	Test that the button starts the settings menu from the main menu	The settings menu starts up	The settings menu starts
Button press "ESC"	Test that the button quitting the game	The game ends	Game stops and shuts down
Button press any other	Tests if any other then expected buttons are pressed	Nothing to happen	Nothing happens

Testing Settings menu:

Function tested	Justification for feature	Expected outcome	Result
Button press "H"	Test that the button starts the help menu from the main menu	Help menu boots up	The help menu starts
Button press "ESC"	Tests that the button results in goes back to the main menu	Main menu starts	Main menu starts
Button press "UP"	Tests that when pressed that the setting selection selects the one below	A change in colour from the one below to show that it is chosen	The setting above is selected and changes colour to yellow
Button press "DOWN"	Tests that when pressed that the setting selection selects the one above	A change in colour from the one above to show that it is chosen	The setting below is selected and changes colour to yellow
Button press "LEFT"	Tests that when the button the number is go down by 20	The settings number to go down	The settings number to go down by 20
Button press "RIGHT"	Tests that when the button the number is go up by 20	The settings number to go up	The settings number to go up by 20
Button press any other	Tests if any other then expected buttons are pressed	Nothing to happen	Nothing happens

Testing Help menu:

Function tested	Justification for feature	Expected outcome	Result
Button press "ESC"	Tests that when pressed the main menu will appear	The main menu will pop up	The main menu started
Button press "BACKSPACE"	Tests that when pressed the game will start	The game to start	The game starts
Button press any other	Tests if any other then expected buttons are pressed	Nothing to happen	Nothing happens

Testing Winner menu:

Function tested	Justification for feature	Expected outcome	Result
Button press "SPACE"	Tests that when pressed the main menu will appear	The main menu will pop up	The main menu started
Button press "BACKSPACE"	Tests that when pressed the game will start	The game to start	The game starts
Button press any other	Tests if any other then expected buttons are pressed	Nothing to happen	Nothing happens

Testing Main game :

Function tested	Justification for feature	Expected outcome	Result
Button press "ESC"	Test that the button quitting the game	The game ends	The game closes
Button press "TAB"	Test that the button qtarts the settings menu from the main menu	The settings menu starts up	The settings menu starts
Button press any other	Tests if any other then expected buttons are pressed	Nothing to happen	Nothing happens
Ball action	Check that the ball will move in the inteded way	When the ball is clicked and draged downwards the ball will react in the oposit way.	Test failed - below is the error and the fix to it

polygons	Tests If the ball hits the breakable polygon	The polygon wall with the appropriate power the it will break	Test failed- below is the error and the fix to it
Walls	Tests if the ball is pulled back and hits the wall	The ball will bounce	Ball does bounce on all 4 sides
counter	If a polygon is broken	The count should go down	It does go down

Error occurred throughout the games development and testing:

While no errors occurred directly in the last version of the game as showed above there were errors in prototypes and previous generations which are showed below.

Regardless some problems did occur in the testing above problems when it came to moving the ball specifically but this could not be fixed.

Error in the original code:

For one during the first edition of the game the function for polygon handling was,

```

81     def handle_polygon_collision(self, arbiter, space, data):
82         impulse = arbiter.total_impulse.length
83         wall_shape = arbiter.shapes[1]
84         for wall in self.polygons:
85             if wall.shape == wall_shape and wall.breakable:
86                 if impulse > wall.destruction_threshold:
87                     space.remove(wall.shape, wall.body)
88                     self.polygons.remove(wall)
89                     break

```

This resulted in the polygon breaking when hitting the floor with any power, The new edition added a new constant and a new interaction, fixing the code.

```

81     def handle_Polygon_collision(arbiter, space, data): #function to handle wall collision
82         if (arbiter.shapes[0].collision_type == COLLTYPE_BALL and #checks if the first shape is a ball
83             arbiter.shapes[1].collision_type == COLLTYPE_POLYGON): #checks if the second shape is a polygon
84
85             impulse = arbiter.total_impulse.length #gets the impulse of the collision (impulse is the force of the collision)
86             wall_shape = arbiter.shapes[1] #gets the shape of the wall
87
88             for wall in polygons: #loops through all the polygons in order to check if the wall is breakable and if the impulse is greater than the destruction threshold
89                 if hasattr(wall, 'shape') and wall.shape == wall_shape and wall.breakable:
90                     if impulse > wall.destruction_threshold:
91                         space.remove(wall.shape, wall.body)
92                         polygons.remove(wall)
93                         break

```

A second problem occurred when the ball was pulled back and It had so much speed that it would glitch out of the space. The distance would be times by a number and unless it was set to a low amount it would glitch out of the space.

```

127         elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
128             if self.selected_ball:
129                 p = Vec2d(event.pos[0], self.h - event.pos[1])
130                 impulse = (self.mouse_start_pos - p) * 30
131                 self.selected_ball.body.apply_impulse_at_local_point(impulse)
132                 self.selected_ball = None
133                 self.mouse_start_pos = None
134             elif event.type == pygame.MOUSEMOTION:
135                 if self.selected_ball:
136                     self.mouse_contact = Vec2d(event.pos[0], self.h - event.pos[1])
137

```

The new code added a maximum of impulse.

```

283         elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
284             if selected_ball:
285                 p = Vec2d(event.pos[0], HEIGHT - event.pos[1])
286                 impulse = (mouse_start_pos - p) * 10
287                 max_impulse = 1000
288                 if impulse.length > max_impulse:
289                     impulse = impulse.normalized() * max_impulse
290                 selected_ball.body.apply_impulse_at_local_point(impulse)
291                 selected_ball = None
292                 mouse_start_pos = None
293             elif event.type == pygame.MOUSEMOTION:
294                 if selected_ball:
295                     mouse_contact = Vec2d(event.pos[0], HEIGHT - event.pos[1])
296

```

Error spotted after testing:

The main error spotted was one with settings; while there was a limit of how small the ball could become there was no limit of how large it could become, same happened with the rest but the ball change made the game unplayable.

```

163     while True:
164         for event in pygame.event.get():
165             if event.type == pygame.QUIT:
166                 return False, None
167             elif event.type == pygame.KEYDOWN:
168                 if event.key == pygame.K_ESCAPE:
169                     return True, MAIN_MENU
170                 elif event.key == pygame.K_UP:
171                     selected_setting = (selected_setting - 1) % len(settings_options)
172                 elif event.key == pygame.K_DOWN:
173                     selected_setting = (selected_setting + 1) % len(settings_options)
174                 elif event.key == pygame.K_LEFT:
175                     settings[settings_options[selected_setting]] = max(100, settings[settings_options[selected_setting]] - 20)
176                 elif event.key == pygame.K_RIGHT:
177                     settings[settings_options[selected_setting]] = settings[settings_options[selected_setting]] + 20
178                 elif event.key == pygame.K_h:
179                     return True, HELP
180

```

The new code fixes this

```

163     while True:
164         for event in pygame.event.get():
165             if event.type == pygame.QUIT:
166                 return False, None
167             elif event.type == pygame.KEYDOWN:
168                 if event.key == pygame.K_ESCAPE:
169                     return True, MAIN_MENU
170                 elif event.key == pygame.K_UP:
171                     selected_setting = (selected_setting - 1) % len(settings_options)
172                 elif event.key == pygame.K_DOWN:
173                     selected_setting = (selected_setting + 1) % len(settings_options)
174                 elif event.key == pygame.K_LEFT:
175                     settings[settings_options[selected_setting]] = max(10, settings[settings_options[selected_setting]] - 20)
176                 elif event.key == pygame.K_RIGHT:
177                     settings[settings_options[selected_setting]] = min(1000, settings[settings_options[selected_setting]] + 20)
178                 elif event.key == pygame.K_h:
179                     return True, HELP
180

```

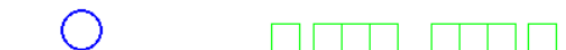
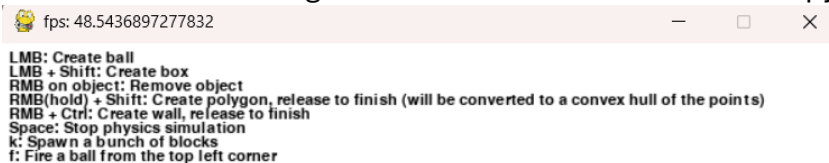
All 5 Stakeholders agreement of development as far:

Signature: _____

Prototypes of the Game:

While the game above shows to be a completed game there was a lot of changes which came about.

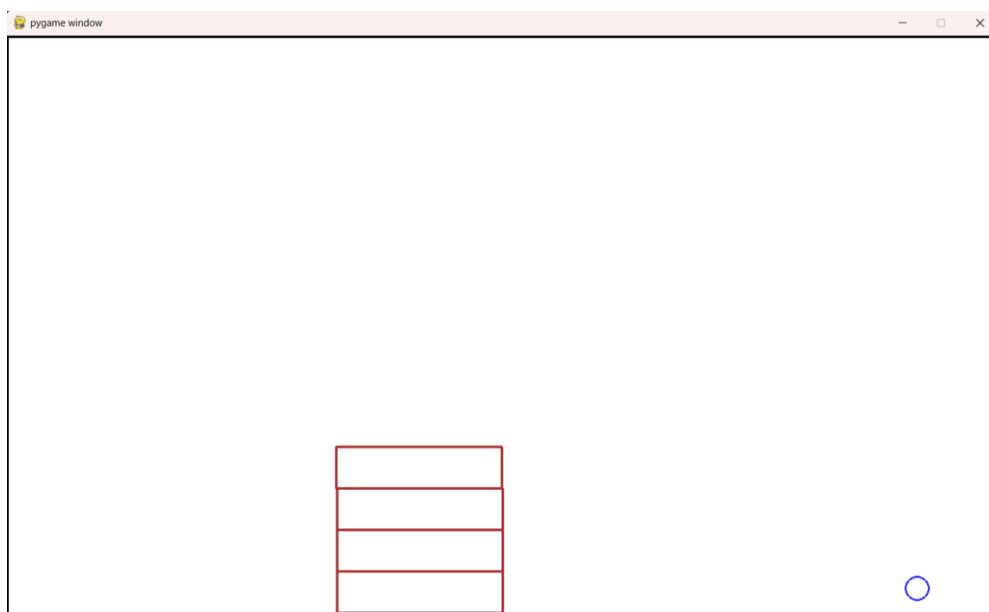
The first edition of the game was created from the demo of pymunk:



This inspired the first version of the game which was simply a ball in the space created.



The game then led to the creation of the breakable polygons.



Which was followed by the creation of the launching ball feature and the Impulse feature and where I saw the first big error in the code, previously mentioned. At this point the game had began to look hi

3.4 Evaluation:

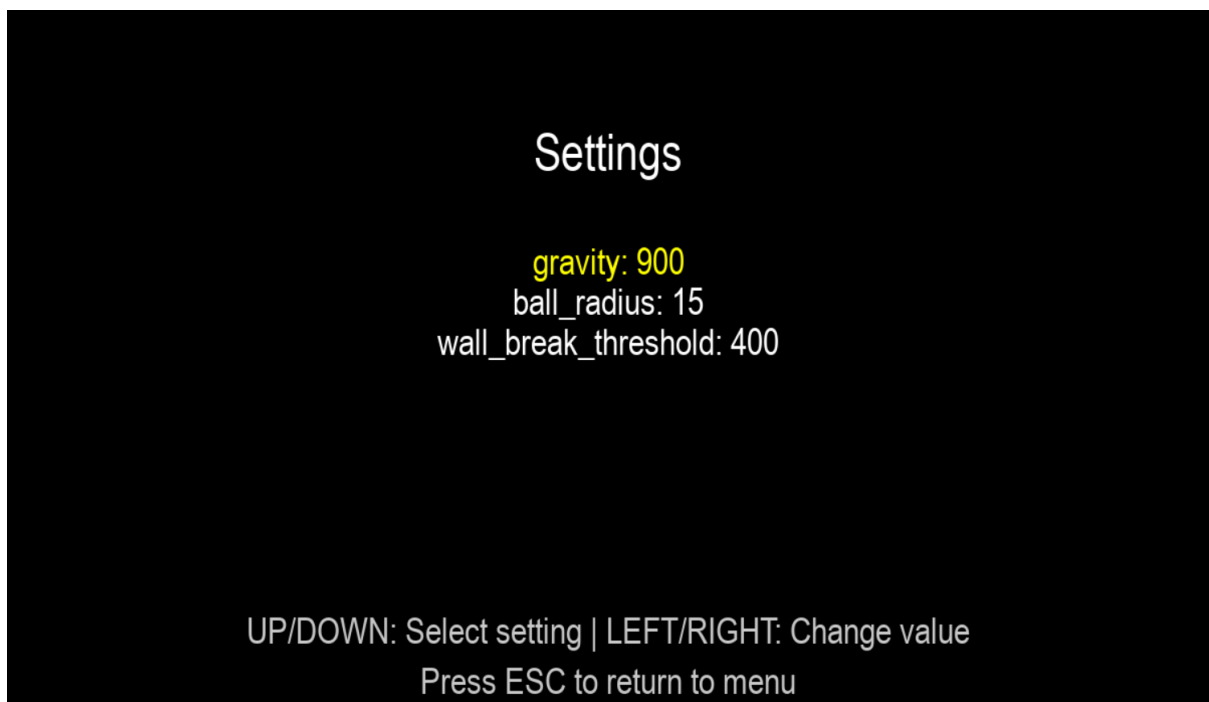
Final look:

The game is now finished as such I have to perform the final tests and reviews with the stakeholders, The testing begins by the booting up of the game;

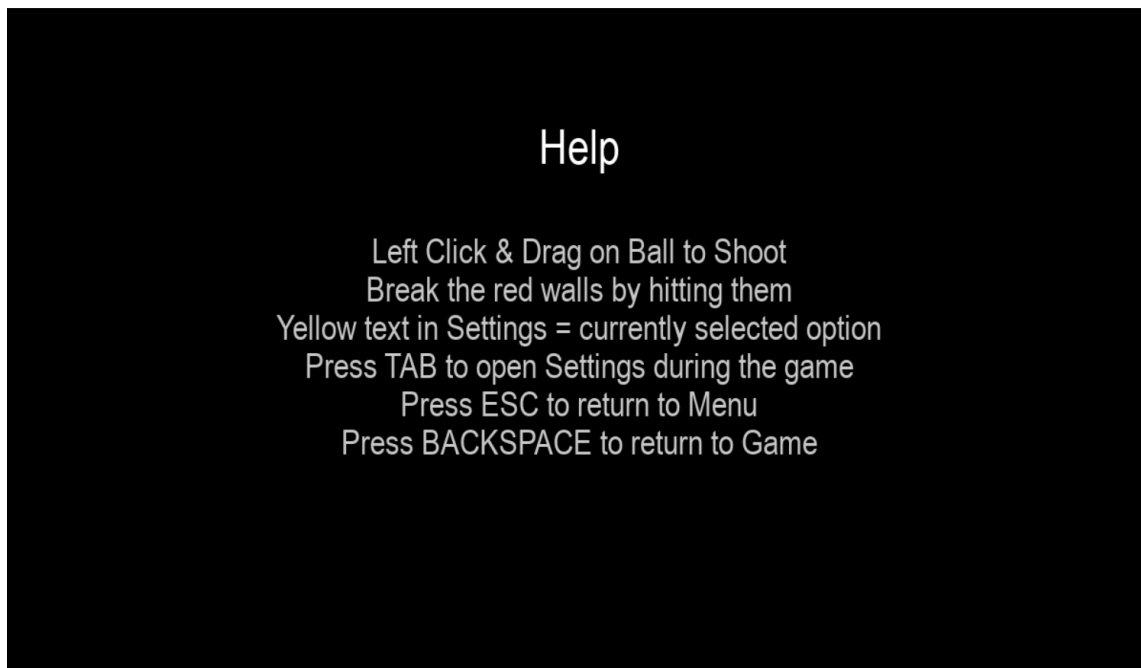


The main menu works on boot up, therefore I continue to test the game by going thru all the buttons and menus

Settings menu:

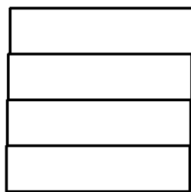


Where I can select the variable to change and how to change it either to increase or decrease. After pressing H the help menu comes up:



Then I tested the game itself, all mechanics of the game works as I fixed the previous problems, one with the blocks breaking when hitting the wall and the other were there so no limiting speed on the ball and therefor it resulted in the ball phasing through the wall of the game and disappearing. But both have now been fixed resulting in a more enjoyable game overall. There was one major problem which I was unable to resolve and it was the integration of a second player, this was due to time restraints and the lack of coding experience as adding a second player would of needed a whole new function in which the players ball cannot affect their own structure and to be ran when the other player finishes their go. Another minor detail which I was not able to integrate was music, this is due to time constraints.

ESC: Menu | TAB: Settings
Breakable Walls Left: 4



For now the game is single player and the blocks which are seen can be broken and can move. It is also possible to create different structures in the programming and to move them around allowing for a more tech savvy user to make their own structures however complex.

When breaking all the block the winner menu will appear:

You Win!

Press SPACE to Play Again
Press ESC to Return to Menu

All 5 Stakeholders agreement of development as far:

Signature: _____

User testing:

After further discussion with the stakeholders about the issues that have been raised with the inability to implement a second player and music an agreement that for now the game is single player and after an update the addition of both can be implemented. They will now play the game themselves and answer questions about their likes and dislikes of the game.

Question	Result	Comment
Is the window the correct size 1200, 700	Answered by all Yes	No comment
Are the colours the ones agreed on	Answered by all Yes	No comment
Do all mechanics like ball launching and block breaking work	Answered by all Yes	The ball is sometimes hard to predict and fidgety.
Do all menus work	Answered by all Yes	No comment
Is the help menu helpful in navigating the game	Answered by all Yes	No comment
Are there any errors or bugs In the game when you played it	Answered by all No	No comment
Are the settings satisfactory	Answered by all Yes	No comment
Is the game less complex then the original	Answered by all Yes	No comment
Is the game more competitive	Answered by all No	Without the integration of a second player the game is more competitive. Agreed by all stakeholders
Are you able to move from menus to menu	Answered by all Yes	No comment
Are all the initial requirements met	3 answered no 2 answered yes	Music and the second player are not included and therefor it dose not meet the original requirements. The second player is a major requirement for the games successes and as such it does not fit the requirements The music is minor but the 2 nd player is significant for the game.
Expect the second player is the game satisfactory	Answered by all Yes	Music is not included but is not necessary

Are the designs of the menu satisfactory	Answered by all Yes	No comment

There was some negative feedback coming from the fact that no second player was included. Nevertheless, all stakeholders were happy with the design colours of the game and the menus alongside the usability of the game mechanics and the game's menus. There was a minor problem where the ball is quite fidgety and unpredictable but overall they all reported it as good.

Features	Evaluation
Assistance	The game has a help menu in order to explain the game and its mechanics
Simplicity	The game is highly simple with shapes and almost no complicated designs.
Consistency	Game play did seem almost repetitive suffering from the same problem as the original game but the simplicity was met with good feedback
Design	Consistent simple design and colors but no music included

Final solution:

Design:

- Medium windows screen 1200, 700 -* requirement met
- The ball will be black -* requirement met
- The blocks will be black -* requirement met
- The background will be white -* requirement met
- The walls will be black -* requirement met
- A noise will be made when the block is broken -* requirement not met
- The ball will need to stay within the windows -* requirement met
- The block will need to stay within the boundaries of the window -* requirement met
- The size of the ball and walls will stay the same unless changed by the settings -* requirement met
- Menus will be created, one for help, main, settings and winner -* requirement met
- The writing will be clear "arial" -* requirement met

Input:

- When the player drags back the ball will launch in the opposite direction
- When the player presses "TAB" the player will be given the main menu screen -* requirement met
- When the player presses "S" the player will be given the Settings menu screen -* requirement met

- when the player presses "H" the player will be given the Help menu screen -* requirement met
 - when in settings the player can change a number which will directly affect the game -* requirement met
 - when the player pulls back from the ball the game should show a small shape -* requirement met
 - when the player presses "SPACE" the game will start -* requirement met
- processes:
- when a ball is broken the countdown must update -* requirement met
 - when all balls are broken the winner menu will pop up -* requirement met
 - menus will show on screen when respected button is pressed -* requirement met
 - The direction of the launch of the ball must be calculated -* requirement met
- Output:
- Correct menus will show when required -* requirement met
 - Noise when the block is broken the number of blocks should update each time the block is broken -* requirement met

All 5 Stakeholders agreement of development as far:

Signature: _____

Usability tested :

Features	Explanation	Evaluation	Met/Not met
Assistance	Without this a new player would become confused and possibly not play the game	The game has a help menu in order to explain the game and its mechanics	Met successfully
Simplicity	By making the game simple it could encourage different people of varying skills to play the game	The game is highly simple with shapes and almost no complicated designs.	Met successfully
Consistency	By having a similar game play it could make the game simpler and mor fun	Game play did seem almost repetitive suffering from the same problem as the original game but the simplicity was met with good feedback	Met successfully
Design	By having an over all theme the player will associate the game with those colors	Consistent simple design and colors but no music included	Met successfully
Competitiveness	By having a second player the game becomes more competitive	Not being able to include this the game does mean it is less competitive	Not met successfully

Success criteria testing:

Criteria	Explanation	Evaluation	Met/Not met
Menus – (help, settings , main , game)	Should be able to move across menus and menus should work as intended with the ability to edit in the settings.	All menus work and you can edit settings along side this you can move through the menus effectively	Met

Ball movement	Should bounce and go in the direction intended the ball should also help velocity to see if when hit if it will break or not.	The ball bounces when hitting walls and polygons	Partially Met
Polygon breaking	Should break when hit with a certain velocity and result in the breaking of it.	The polygons break when hit with the ball	Met
Difficulty	Game should be winnable but not too easy show present some challenge	The game does not have a losing feature but you can win	Partially met
Lag	Should not take too long to load up the game, or when hitting a shape(breakable polygon) it should not lag .	The game does not lag and does not take long to load	Met
Responsiveness / Controls	The game should respond to user input and button presses responding in a timely manner.	The game responds well and effectively	Met
Music/noise control	Music/noise should be able to be changed in terms of volume and if its on or off and when a noise activates it should do it in a timely order.	Music and noise was not included in the final game	Not Met
Powerups working	The ball should react when coming into contact the powerups and should do it without lag	The power ups were not included in the final game	Not Met

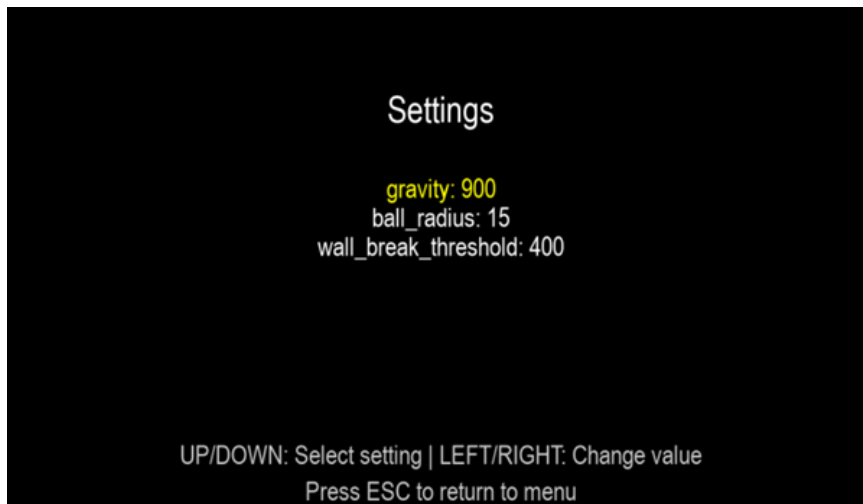
Final game design evaluation :

design evaluation Main menu:



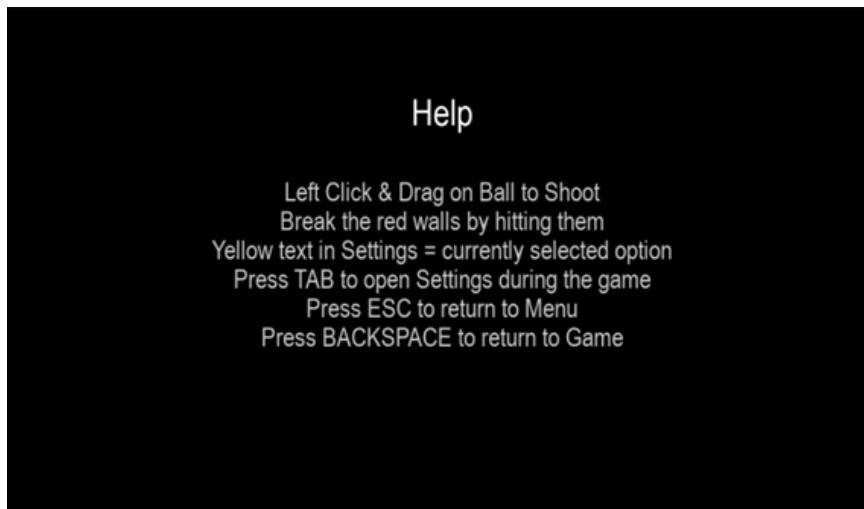
Highly simple with black background and white writing and instructions how to advance, with the title larger than everything else.

design evaluation Settings menu:



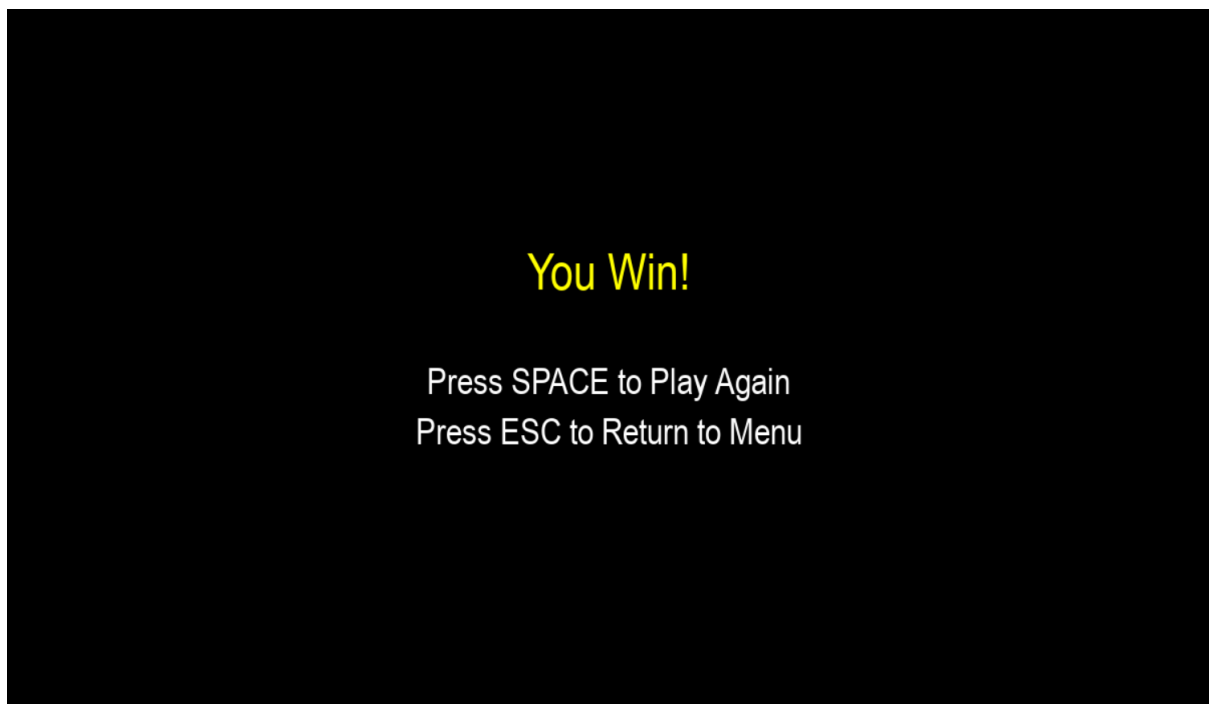
Settings allows you to change and has a real impact on the game like the ball size with a yellow underline on the setting that you choose to edit to show that you are editing that setting

design evaluation Help menu:



This is instructional for those that have not played the game before where it explains everything on a clear white font on a black background.

design evaluation Winner menu:



Instructs you how to advance and replay if the person wishes, a clear statement that the game is over and that the player is a winner.

Maintenance and development:

In the game futures I need to continue maintain the game as without consistent updates the software it is ran on like python and pygame and pymunk will update and leave the game with possible bugs and freezes allowing the game to run smoothly.

The game has high potential for brining new features in the future, like the 2nd player which can be added with more time nevertheless the game is enjoyable with single player.

Along the same lines another problem the game has is its inability to lose, there is no way for the user to not win eventually unless there is an error. This is a problem as it goes against the goal of the game to be competitive. As there is no way to fail at the game

Pros of the game are

- Easy to play and understand
- Simple
- Runs smoothly without lag and bugs

Cons of the game are

- No second player
- Limiting in design as it would be difficult to implement designs for the menus
- No way to lose

Ball action	Check that the ball will move in the inteded way	When the ball is clicked and draged downwards the ball will react in the oposit way.	Test failed- below is the error and the fix to it
polygons	Tests If the ball hits the breakable polygon	The polygon wall with the apropeate power the it will break	Test failed- below is the error and the fix to it

Two tests failed and were fixed in the end there was no testing which was not able to be resolved.

Success criteria later development:

Criteria	Met/Not met	Later development
Menus – (help, settings , main , game)	Met	All menus work and moving across them can be done
Ball movement	Partially Met	The ball has some problems with the direction when launching it
Polygon breaking	Met	The polygons break when hit

Difficulty	Partially met	Introducing a second player would make the game significantly harder and give the ability to lose
Lag	Met	There is no lag at all throughout the testing
Responsiveness / Controls	Met	When pressing any button the game is reactive and it occurs in seconds
Music/noise control	Not Met	Introduction of music and noise when the ball hits the ground would fill this criteria.
Powerups working	Not Met	Introducing powerups would fulfill this criteria and making it when the ball hits the powerup an effect is produced

Usability tested later development:

Features	Met/Not met	
Assistance	Met successfully	The game if not innate has help at all menus on how the buttons when and the game works like in the help menu
Simplicity	Met successfully	The game is simple and not hard to understand
Consistency	Met successfully	All menus and game looks the same, the game although play is repetitive and it stays the same without changing until a setting is changed, in that case the game play is different but mechanics are the same
Design	Met successfully	Has an overall theme in the appearance of the game
Competitiveness	Not met successfully	Introducing a second player would make the game significantly harder and give the ability to lose